C++ Standard Library Issues to be moved in Virtual Plenary, Oct. 2021

Doc. no. P2450R0

Date: 2021-09-24

Audience: WG21
Reply to: Jonathan Wakely <lwgchair@gmail.com>

Tentatively Ready Issues

2191. Incorrect specification of `match_results(match_results&&)`

Section: 30.9.2 [re.results.const] **Status:** Tentatively Ready **Submitter:** Pete Becker **Opened:** 2012-10-02 **Last modified:** 2021-07-17

Priority: 4

View all other issues in [re.results.const].

Discussion:

30.9.2 [re.results.const]/3: "Move-constructs an object of class `match_results` satisfying the same postconditions as Table 141."

Table 141 lists various member functions and says that their results should be the results of the corresponding member function calls on `m`. But `m` has been moved from, so the actual requirement ought to be based on the value that `m` had *before* the move construction, not on `m` itself.

In addition to that, the requirements for the copy constructor should refer to Table 141.

Ganesh:

Also, the requirements for move-assignment should refer to Table 141. Further it seems as if in Table 141 all phrases of "for all integers `n < m.size()`" should be replaced by "for all *unsigned* integers `n < m.size()`".

[2019-03-26; Daniel comments and provides wording]

The previous Table 141 (Now Table 128 in N4810) has been modified to cover now the effects of move/copy constructors and move/copy assignment operators. Newly added wording now clarifies that for move operations the corresponding values refer to the values of the move source *before* the operation has started.

Re Ganesh's proposal: Note that no further wording is needed for the move-assignment operator, because in the current working draft the move-assignment operator's *Effects:* element refers already to Table 128. The suggested clarification of *unsigned* integers has been implemented by referring to *non-negative* integers instead.

Upon suggestion from Casey, the wording also introduces *Ensures:* elements that refer to Table 128 and as drive-by fix eliminates a "*Throws:* Nothing." element from a `noexcept` function.

Previous resolution [SUPERSEDED]:

This wording is relative to N4810.

1. Add a new paragraph at the beginning of 30.9.2 [re.results.const] as indicated:

   -?- Table 128 lists the postconditions of `match_results` copy/move constructors and copy/move assignment operators. For move operations, the results of the expressions depending on the parameter `m` denote the values they had before the respective function calls.

2. Modify 30.9.2 [re.results.const] as indicated:

```
match_results(const match_results& m);
```

   -3- *Effects:* Constructs ~~an object of class `match_results`, as~~ a copy of `m`.

   -?- *Ensures:* As indicated in Table 128.

```
match_results(match_results&& m) noexcept;
```

-4- *Effects:* Move constructs ~~an object of class `match_results`~~ from `m` ~~satisfying the same postconditions as Table 128. Additionally~~, the stored `Allocator` value is move constructed from `m.get_allocator()`.

-?- *Ensures:* As indicated in Table 128.

~~-5- *Throws:* Nothing.~~

```
match_results& operator=(const match_results& m);
```

-6- *Effects:* Assigns `m` to `*this`. ~~The postconditions of this function are indicated in Table 128.~~

-?- *Ensures:* As indicated in Table 128.

```
match_results& operator=(match_results&& m);
```

-7- *Effects:* Move~~-~~ assigns `m` to `*this`. ~~The postconditions of this function are indicated in Table 128.~~

-?- *Ensures:* As indicated in Table 128.

3. Modify 30.9.2 [re.results.const], Table 128 — "`match_results` assignment operator effects", as indicated:

Table 128 — `match_results` ~~assignment operator effects~~copy/move operation postconditions

Element	Value
`ready()`	`m.ready()`
`size()`	`m.size()`
`str(n)`	`m.str(n)` for all non-negative integers `n < m.size()`
`prefix()`	`m.prefix()`
`suffix()`	`m.suffix()`
`(*this)[n]`	`m[n]` for all non-negative integers `n < m.size()`
`length(n)`	`m.length(n)` for all non-negative integers `n < m.size()`
`position(n)`	`m.position(n)` for all non-negative integers `n < m.size()`

[2021-06-25; Daniel comments and provides new wording]

The revised wording has been rebased to N4892. It replaces the now obsolete *Ensures:* element by the *Postconditions:* element and also restores the copy constructor prototype that has been eliminated by P1722R2 as anchor point for the link to Table 140.

[2021-06-30; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Add a new paragraph at the beginning of 30.9.2 [re.results.const] as indicated:

-?- Table 140 [tab:re.results.const] lists the postconditions of `match_results` copy/move constructors and copy/move assignment operators. For move operations, the results of the expressions depending on the parameter `m` denote the values they had before the respective function calls.

2. Modify 30.9.2 [re.results.const] as indicated:

```
explicit match_results(const Allocator& a);
```

-1- *Postconditions:* `ready()` returns `false`. `size()` returns `0`.

```
match_results(const match_results& m);
```

-?- *Postconditions:* As specified in Table 140.

```
match_results(match_results&& m) noexcept;
```

-2- *Effects:* The stored `Allocator` value is move constructed from `m.get_allocator()`.

-3- *Postconditions:* As specified in Table 140.

```
match_results& operator=(const match_results& m);
```

-4- *Postconditions:* As specified in Table 140.

```
match_results& operator=(match_results&& m);
```

-5- *Postconditions:* As specified in Table 140.

3. Modify 30.9.2 [re.results.const], Table 140 — "`match_results` assignment operator effects", [tab:re.results.const], as indicated:

Table 140 — `match_results` ~~assignment operator effects~~copy/move operation postconditions

Element	Value
`ready()`	`m.ready()`
`size()`	`m.size()`
`str(n)`	`m.str(n)` for all non-negative integers `n < m.size()`
`prefix()`	`m.prefix()`
`suffix()`	`m.suffix()`
`(*this)[n]`	`m[n]` for all non-negative integers `n < m.size()`
`length(n)`	`m.length(n)` for all non-negative integers `n < m.size()`
`position(n)`	`m.position(n)` for all non-negative integers `n < m.size()`

2381. Inconsistency in parsing floating point numbers

Section: 28.4.3.2.3 [facet.num.get.virtuals] **Status:** Tentatively Ready **Submitter:** Marshall Clow **Opened:** 2014-04-30 **Last modified:** 2021-09-20

Priority: 2

View other active issues in [facet.num.get.virtuals].

View all other issues in [facet.num.get.virtuals].

Discussion:

In 28.4.3.2.3 [facet.num.get.virtuals] we have:

> Stage 3: The sequence of chars accumulated in stage 2 (the field) is converted to a numeric value by the rules of one of the functions declared in the header `<cstdlib>`:
>
> - For a signed integer value, the function `strtoll`.
> - For an unsigned integer value, the function `strtoull`.
> - For a floating-point value, the function `strtold`.

This implies that for many cases, this routine should return true:

```
bool is_same(const char* p)
{
  std::string str{p};
  double val1 = std::strtod(str.c_str(), nullptr);
  std::stringstream ss(str);
  double val2;
  ss >> val2;
  return std::isinf(val1) == std::isinf(val2) &&                // either they're both infinity
         std::isnan(val1) == std::isnan(val2) &&                // or they're both NaN
         (std::isinf(val1) || std::isnan(val1) || val1 == val2); // or they're equal
}
```

and this is indeed true, for many strings:

```
assert(is_same("0"));
assert(is_same("1.0"));
assert(is_same("-1.0"));
assert(is_same("100.123"));
assert(is_same("1234.456e89"));
```

but not for others

```
assert(is_same("0xABp-4")); // hex float
assert(is_same("inf"));
assert(is_same("+inf"));
assert(is_same("-inf"));
assert(is_same("nan"));
assert(is_same("+nan"));
assert(is_same("-nan"));
```

```
assert(is_same("infinity"));
assert(is_same("+infinity"));
assert(is_same("-infinity"));
```

These are all strings that are correctly parsed by `std::strtod`, but not by the stream extraction operators. They contain characters that are deemed invalid in stage 2 of parsing.

If we're going to say that we're converting by the rules of `strtold`, then we should accept all the things that `strtold` accepts.

[2016-04, Issues Telecon]

People are much more interested in round-tripping hex floats than handling `inf` and `nan`. Priority changed to P2.

Marshall says he'll try to write some wording, noting that this is a very closely specified part of the standard, and has remained unchanged for a long time. Also, there will need to be a sample implementation.

[2016-08, Chicago]

Zhihao provides wording

The `src` array in Stage 2 does narrowing only. The actual input validation is delegated to `strtold` (independent from the parsing in Stage 3 which is again being delegated to `strtold`) by saying:

[...] If it is not discarded, then a check is made to determine if c is allowed as the next character of an input field of the conversion specifier returned by Stage 1.

So a conforming C++11 `num_get` is supposed to magically accept an hexfloat without an exponent

0x3.AB

because we refers to C99, and the fix to this issue should be just expanding the `src` array.

Support for Infs and NaNs are not proposed because of the complexity of nan(n-chars).

[2016-08, Chicago]

Tues PM: Move to Open

[2016-09-08, Zhihao Yuan comments and updates proposed wording]

Examples added.

[2018-08-23 Batavia Issues processing]

Needs an Annex C entry. Tim to write Annex C.

Previous resolution [SUPERSEDED]:

> This wording is relative to N4606.
>
> 1. Change 28.4.3.2.3 [facet.num.get.virtuals]/3 Stage 2 as indicated:
>
> ```
> static const char src[] = "0123456789abcdefpxABCDEFPX+-";
> ```
>
> 2. Append the following examples to 28.4.3.2.3 [facet.num.get.virtuals]/3 Stage 2 as indicated:
>
> [*Example:*
>
> Given an input sequence of `"0x1a.bp+07p"`,
>
> - if Stage 1 returns `%d`, `"0"` is accumulated;
> - if Stage 1 returns `%i`, `"0x1a"` are accumulated;
> - if Stage 1 returns `%g`, `"0x1a.bp+07"` are accumulated.
>
> In all cases, leaving the rest in the input.
>
> — end example]

[2021-05-18 Tim updates wording]

Based on the git history, libc++ appears to have always included `p` and `P` in `src`.

[2021-09-20; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Change 28.4.3.2.3 [facet.num.get.virtuals]/3 Stage 2 as indicated:

> — Stage 2:
>
> If `in == end` then stage 2 terminates. Otherwise a `charT` is taken from `in` and local variables are initialized as if by
>
> ```
> char_type ct = *in;
> char c = src[find(atoms, atoms + sizeof(src) - 1, ct) - atoms];
> if (ct == use_facet<numpunct<charT>>(loc).decimal_point())
> c = '.';
> bool discard =
>   ct == use_facet<numpunct<charT>>(loc).thousands_sep()
>   && use_facet<numpunct<charT>>(loc).grouping().length() != 0;
> ```
>
> where the values `src` and `atoms` are defined as if by:
>
> ```
> static const char src[] = "0123456789abcdefpxABCDEFPX+-";
> char_type atoms[sizeof(src)];
> use_facet<ctype<charT>>(loc).widen(src, src + sizeof(src), atoms);
> ```
>
> for this value of `loc`.
>
> If `discard` is true, then if `'.'` has not yet been accumulated, then the position of the character is remembered, but the character is otherwise ignored. Otherwise, if `'.'` has already been accumulated, the character is discarded and Stage 2 terminates. If it is not discarded, then a check is made to determine if `c` is allowed as the next character of an input field of the conversion specifier returned by Stage 1. If so, it is accumulated.
>
> If the character is either discarded or accumulated then `in` is advanced by `++in` and processing returns to the beginning of stage 2.
>
> [*Example:*
>
> Given an input sequence of `"0x1a.bp+07p"`,
>
> - if the conversion specifier returned by Stage 1 is `%d`, `"0"` is accumulated;
> - if the conversion specifier returned by Stage 1 is `%i`, `"0x1a"` are accumulated;
> - if the conversion specifier returned by Stage 1 is `%g`, `"0x1a.bp+07"` are accumulated.
>
> In all cases, the remainder is left in the input.
>
> — end example]

2. Add the following new subclause to C.5 [diff.cpp03]:

> **C.4.? [locale]: localization library [diff.cpp03.locale]**
>
> **Affected subclause:** 28.4.3.2.3 [facet.num.get.virtuals]
> **Change:** The `num_get` facet recognizes hexadecimal floating point values.
> **Rationale:** Required by new feature.
> **Effect on original feature:** Valid C++2003 code may have different behavior in this revision of C++.

2762. `unique_ptr operator*()` should be `noexcept`

Section: 20.11.1.3.5 [unique.ptr.single.observers] **Status:** Tentatively Ready **Submitter:** Ville Voutilainen **Opened:** 2016-08-04 **Last modified:** 2021-06-23

Priority: 3

View all other issues in [unique.ptr.single.observers].

Discussion:

See LWG 2337. Since we aren't removing `noexcept` from `shared_ptr`'s `operator*`, we should consider adding `noexcept` to `unique_ptr`'s `operator*`.

[2016-08 — Chicago]

Thurs PM: P3, and status to 'LEWG'

[2016-08-05 Chicago]

Ville provides an initial proposed wording.

[LEWG Kona 2017]

->Open: Believe these should be `noexcept` for consistency. We like these. We agree with the proposed resolution. `operator->()` already has `noexcept`.

Also adds `optional::operator*`

Alisdair points out that fancy pointers might intentionally throw from `operator*`, and we don't want to prohibit that.

Go forward with conditional `noexcept(noexcept(*decltype()))`.

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

[Drafting note: since this issue is all about consistency, `optional`'s pointer-like operators are additionally included.]

1. In 20.11.1.3 [unique.ptr.single] synopsis, edit as follows:

```
add_lvalue_reference_t<T> operator*() const noexcept;
```

2. Before 20.11.1.3.5 [unique.ptr.single.observers]/1, edit as follows:

```
add_lvalue_reference_t<T> operator*() const noexcept;
```

3. In 20.6.3 [optional.optional] synopsis, edit as follows:

```
constexpr T const *operator->() const noexcept;
constexpr T *operator->() noexcept;
constexpr T const &operator*() const & noexcept;
constexpr T &operator*() & noexcept;
constexpr T &&operator*() && noexcept;
constexpr const T &&operator*() const && noexcept;
```

4. Before [optional.object.observe]/1, edit as follows:

```
constexpr T const* operator->() const noexcept;
constexpr T* operator->() noexcept;
```

5. Before [optional.object.observe]/5, edit as follows:

```
constexpr T const& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
```

6. Before [optional.object.observe]/9, edit as follows:

```
constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const && noexcept;
```

[2021-06-19 Tim updates wording]

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

[Drafting note: since this issue is all about consistency, `optional`'s pointer-like operators are additionally included.]

1. Edit 20.11.1.3.1 [unique.ptr.single.general], class template `unique_ptr` synopsis, as indicated:

```
namespace std {
  template<class T, class D = default_delete<T>> class unique_ptr {
  public:
```

```
    […]

    // 20.11.1.3.5 [unique.ptr.single.observers], observers
    add_lvalue_reference_t<T> operator*() const noexcept(see below);
    […]
  };
}
```

2. Edit 20.11.1.3.5 [unique.ptr.single.observers] as indicated:

```
add_lvalue_reference_t<T> operator*() const noexcept(noexcept(*declval<pointer>()));
```

-1- *Preconditions*: `get() != nullptr`.

-2- *Returns*: `*get()`.

3. Edit 20.6.3.1 [optional.optional.general], class template `optional` synopsis, as indicated:

```
namespace std {
  template<class T>
  class optional {
  public:
    […]

    // 20.6.3.6 [optional.observe], observers
    constexpr const T* operator->() const noexcept;
    constexpr T* operator->() noexcept;
    constexpr const T& operator*() const & noexcept;
    constexpr T& operator*() & noexcept;
    constexpr T&& operator*() && noexcept;
    constexpr const T&& operator*() const && noexcept;

    […]
  };
}
```

4. Edit 20.6.3.6 [optional.observe] as indicated:

```
constexpr const T* operator->() const noexcept;
constexpr T* operator->() noexcept;
```

-1- *Preconditions*: `*this` contains a value.

-2- *Returns*: `val`.

~~-3- *Throws*: Nothing.~~

-4- *Remarks:* These functions are constexpr functions.

```
constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
```

-5- *Preconditions*: `*this` contains a value.

-6- *Returns*: `*val`.

~~-7- *Throws*: Nothing.~~

-8- *Remarks:* These functions are constexpr functions.

```
constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const && noexcept;
```

-9- *Preconditions*: `*this` contains a value.

-10- *Effects*: Equivalent to: `return std::move(*val);`

3121. `tuple` constructor constraints for `UTypes&&...` overloads

Section: 20.5.3.1 [tuple.cnstr] **Status:** Tentatively Ready **Submitter:** Matt Calabrese **Opened:** 2018-06-12 **Last modified:** 2021-05-21

Priority: 2

View other active issues in [tuple.cnstr].

View all other issues in [tuple.cnstr].

Discussion:

Currently the `tuple` constructors of the form:

```
template<class... UTypes>
EXPLICIT constexpr tuple(UTypes&&...);
```

are not properly constrained in that in the 1-element `tuple` case, the constraints do no short-circuit when the constructor would be (incorrectly) considered as a possible copy/move constructor candidate. libc++ has a workaround for this, but the additional short-circuiting does not actually appear in the working draft.

As an example of why this lack of short circuiting is a problem in practice, consider the following line:

```
bool a = std::is_copy_constructible_v<std::tuple<any>>;
```

The above code will cause a compile error because of a recursive trait definition. The copy constructibility check implies doing substitution into the `UTypes&&...` constructor overloads, which in turn will check if `tuple<any>` is convertible to any, which in turn will check if `tuple<any>` is copy constructible (and so the trait is dependent on itself).

I do not provide wording for the proposed fix in anticipation of requires clauses potentially changing how we do the specification, however, the basic solution should be similar to what we've done for other standard library types, which is to say that the very first constraint should be to check that if `sizeof...(UTypes) == 1` and the type, after applying `remove_cvref_t`, is the `tuple` type itself, then we should force substitution failure rather than checking any further constraints.

[2018-06-23 after reflector discussion]

Priority set to 3

[2018-08-20, Jonathan provides wording]

[2018-08-20, Daniel comments]

The wording changes by this issue are very near to those suggested for LWG 3155.

[2018-11 San Diego Thursday night issue processing]

Jonathan to update wording - using conjunction. Priority set to 2

Previous resolution [SUPERSEDED]:

This wording is relative to N4762.

- Modify 20.5.3.1 [tuple.cnstr] as indicated:

```
template<class... UTypes> explicit(see below) constexpr tuple(UTypes&&... u);
```

-9- *Effects:* Initializes the elements in the tuple with the corresponding value in `std::forward<UTypes>(u)`.

-10- *Remarks:* This constructor shall not participate in overload resolution unless `sizeof...(Types) == sizeof...(UTypes)` and `sizeof...(Types) >= 1` and `(sizeof...(Types) > 1 || !is_same_v<remove_cvref_t<U0>, tuple>)` and `is_constructible_v<Ti, Ui&&>` is `true` for all *i*. The expression inside `explicit` is equivalent to:

```
!conjunction_v<is_convertible<UTypes, Types>...>
```

[2021-05-20 Tim updates wording]

The new wording below also resolves LWG 3155, relating to an `allocator_arg_t` tag argument being treated by this constructor template as converting to the first tuple element instead of as a tag. To minimize collateral damage, this wording takes this constructor out of overload resolution only if the tuple is of size 2 or 3, the first argument is an `allocator_arg_t`, but the first tuple element isn't of type `allocator_arg_t` (in both cases after removing cv/ref qualifiers). This avoids damaging tuples that actually contain an `allocator_arg_t` as the first element (which can be formed during uses-allocator construction, thanks to `uses_allocator_construction_args`).

The proposed wording has been implemented and tested on top of libstdc++.

[2021-08-20; LWG telecon]

Set status to Tentatively Ready after telecon review.

Proposed resolution:

This wording is relative to N4885, and also resolves LWG 3155.

1. Modify 20.5.3.1 [tuple.cnstr] as indicated:

`template<class... UTypes> explicit(`*see below*`) constexpr tuple(UTypes&&... u);`

-?- Let *disambiguating-constraint* be:

(?.1) — `negation<is_same<remove_cvref_t<U0>, tuple>>` if `sizeof...(Types)` is 1;

(?.2) — otherwise, `bool_constant<!is_same_v<remove_cvref_t<U0>, allocator_arg_t> || is_same_v<remove_cvref_t<T0>, allocator_arg_t>>` if `sizeof...(Types)` is 2 or 3;

(?.3) — otherwise, `true_type`.

-12- *Constraints:*

(12.1) — `sizeof...(Types)` equals `sizeof...(UTypes)`, ~~and~~

(12.2) — `sizeof...(Types)` ≥ 1, and

(12.3) — `conjunction_v<`*disambiguating-constraint*`, is_constructible<Types, UTypes>...>` is true ~~`is_constructible_v<Ti, Ui>` is true for all i~~.

-13- *Effects:* Initializes the elements in the tuple with the corresponding value in `std::forward<UTypes>(u)`.

-14- *Remarks:* The expression inside `explicit` is equivalent to:

```
!conjunction_v<is_convertible<UTypes, Types>...>
```

3123. `duration` constructor from representation shouldn't be effectively non-throwing

Section: 27.5 [time.duration] **Status:** Tentatively Ready **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2018-06-22 **Last modified:** 2021-09-20

Priority: 3

View all other issues in [time.duration].

Discussion:

[time.duration]/4 states:

> Members of duration shall not throw exceptions other than those thrown by the indicated operations on their representations.

Where representation is defined in the non-normative, brief description at [time.duration]/1:

> […] A duration has a representation which holds a count of ticks and a tick period. […]

[time.duration.cons]/2 doesn't indicate the operation undergone by its representation, merely stating a postcondition in [time.duration.cons]/3:

> *Effects:* Constructs an object of type `duration`.
>
> *Postconditions:* `count() == static_cast<rep>(r)`.

I suggest this reformulation that follows the format of [time.duration.cons]/5.

> *Effects:* Constructs an object of type duration, constructing `rep_` from `r`.

Now it is clear why the constructor would throw.

Previous resolution [SUPERSEDED]:

This wording is relative to N4750.

Change 27.5.2 [time.duration.cons] as indicated:

```
template<class Rep2>
  constexpr explicit duration(const Rep2& r);
```

-1- *Remarks:* This constructor shall not participate in overload resolution unless […]

-2- *Effects:* Constructs an object of type `duration`, constructing `rep_` from `r`.

-3- *Postconditions:* `count() == static_cast<rep>(r)`.

[2018-06-27 after reflector discussion]

Priority set to 3. Improved wording as result of that discussion.

Previous resolution [SUPERSEDED]:

> This wording is relative to N4750.
>
> Change 27.5.2 [time.duration.cons] as indicated:
>
> ```
> template<class Rep2>
>   constexpr explicit duration(const Rep2& r);
> ```
>
> -1- *Remarks:* This constructor shall not participate in overload resolution unless […]
>
> -2- *Effects:* ~~Constructs an object of type `duration`~~Initializes `rep_` with `r`.
>
> ~~-3- *Postconditions:* `count() == static_cast<rep>(r)`.~~

[2020-05-02; Daniel resyncs wording with recent working draft]

[2021-09-20; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4861.

Change 27.5.2 [time.duration.cons] as indicated:

```
template<class Rep2>
  constexpr explicit duration(const Rep2& r);
```

-1- *Constraints:* `is_convertible_v<const Rep2&, rep>` is `true` and

(1.1) — `treat_as_floating_point_v<rep>` is `true` or

(1.2) — `treat_as_floating_point_v<Rep2>` is `false`.

[*Example:* […] *end example*]

-?- *Effects:* Initializes `rep_` with `r`.

~~-2- *Postconditions:* `count() == static_cast<rep>(r)`.~~

3146. Excessive unwrapping in `std::ref`/`cref`

Section: 20.14.6.6 [refwrap.helpers] **Status:** Tentatively Ready **Submitter:** Agustín K-ballo Bergé **Opened:** 2018-07-10 **Last modified:** 2021-05-22

Priority: 3

Discussion:

The overloads of `std::ref`/`cref` that take a `reference_wrapper` as argument are defined as calling `std::ref`/`cref` recursively, whereas the return type is defined as unwrapping just one level. Calling these functions with arguments of multiple level of wrapping leads to ill-formed programs:

```
int i = 0;
std::reference_wrapper<int> ri(i);
std::reference_wrapper<std::reference_wrapper<int>> rri(ri);
std::ref(rri); // error within 'std::ref'
```

[Note: these overloads were added by issue resolution 10.29 for TR1, which can be found at N1688, at Redmond 2004]

[2018-08-20 Priority set to 3 after reflector discussion]

[2021-05-22 Tim syncs wording to the current working draft]

[2021-08-20; LWG telecon]

Set status to Tentatively Ready after telecon review.

Proposed resolution:

This wording is relative to N4885.

1. Change 20.14.6.6 [refwrap.helpers] as indicated:

```
template<class T> constexpr reference_wrapper<T> ref(reference_wrapper<T> t) noexcept;
```

-3- *Returns:* ~~`ref(t.get())`~~`t`.

[…]

```
template<class T> constexpr reference_wrapper<const T> cref(reference_wrapper<T> t) noexcept;
```

-5- *Returns:* ~~`cref(t.get())`~~`t`.

3152. `common_type` and `common_reference` have flaws in common

Section: 20.15.8.7 [meta.trans.other] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2018-08-10 **Last modified:** 2021-06-23

Priority: 3

View other active issues in [meta.trans.other].

View all other issues in [meta.trans.other].

Discussion:

20.15.8.7 [meta.trans.other] p5 characterizes the requirements for program-defined specializations of `common_type` with the sentence:

> Such a specialization need not have a member named `type`, but if it does, that member shall be a *typedef-name* for an accessible and unambiguous cv-unqualified non-reference type `C` to which each of the types `T1` and `T2` is explicitly convertible.

This sentence - which 20.15.8.7 [meta.trans.other] p7 largely duplicates to specify requirements on program-defined specializations of `basic_common_reference` - has two problems:

1. The grammar term "*typedef-name*" is overconstraining; there's no reason to prefer a *typedef-name* here to an actual type, and
2. "accessible" and "unambiguous" are not properties of *types*, they are properties of names and base classes.
3. While we're here, we may as well strike the unused name `C` which both Note B and Note D define for the type denoted by `type`.

[2018-08 Batavia Monday issue prioritization]

Priority set to 3

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4762.

1. Modify 20.15.8.7 [meta.trans.other] p5 as follows:

> -5- Note B: Notwithstanding the provisions of 20.15.3 [meta.type.synop], and pursuant to 16.4.5.2.1 [namespace.std], a program may specialize `common_type<T1, T2>` for types `T1` and `T2` such that `is_same_v<T1, decay_t<T1>>` and `is_same_v<T2, decay_t<T2>>` are each true. [*Note:* …] Such a specialization need not have a member named `type`, but if it does, ~~that member shall be a *typedef-name* for an accessible and unambiguous~~ the *qualified-id* `common_type<T1, T2>::type` shall denote a cv-unqualified non-reference type ~~`C`~~ to which each of the types `T1` and `T2` is explicitly convertible. Moreover, […]

2. Modify 20.15.8.7 [meta.trans.other] p7 similarly:

> -7- Note D: Notwithstanding the provisions of 20.15.3 [meta.type.synop], and pursuant to 16.4.5.2.1 [namespace.std], a program may partially specialize `basic_common_reference<T, U, TQual, UQual>` for types `T` and `U` such that `is_same_v<T, decay_t<T>>` and `is_same_v<U, decay_t<U>>` are each true. [*Note:* …] Such a specialization need not have a member named `type`, but if it does, ~~that member shall be a *typedef-name* for an accessible and unambiguous~~ the *qualified-id* `basic_common_reference<T, U, TQual, UQual>::type` shall denote a cv-unqualified non-reference type ~~`C`~~ to which each of the types `TQual<T>` and `UQual<U>` is convertible. Moreover, […]

3293. `move_iterator operator+()` has incorrect constraints

Section: 23.5.3.9 [move.iter.nonmember] **Status:** Tentatively Ready **Submitter:** Bo Persson **Opened:** 2019-09-13 **Last modified:** 2021-06-23

Priority: 3

View all other issues in [move.iter.nonmember].

Discussion:

Section 23.5.3.9 [move.iter.nonmember]/2-3 says:

```
template<class Iterator>
  constexpr move_iterator<Iterator>
    operator+(iter_difference_t<Iterator> n, const move_iterator<Iterator>& x);
```

Constraints: `x + n` is well-formed and has type `Iterator`.

Returns: `x + n`.

However, the return type of this operator is `move_iterator<Iterator>`, so the expression `x + n` ought to have that type. Also, there is no `operator+` that matches the constraints, so it effectively disables the addition.

[2019-10-31 Issue Prioritization]

Priority to 3 after reflector discussion.

Previous resolution [SUPERSEDED]:

This wording is relative to N4830.

1. Modify 23.5.3.9 [move.iter.nonmember] as indicated:

```
template<class Iterator>
  constexpr move_iterator<Iterator>
    operator+(iter_difference_t<Iterator> n, const move_iterator<Iterator>& x);
```

-2- *Constraints:* `x + n` is well-formed and has type `move_iterator<Iterator>`.

-3- *Returns:* `x + n`.

[2019-11-04; Casey comments and provides revised wording]

After applying the P/R the *Constraint* element requires `x + n` to be well-formed (it always is, since that operation is unconstrained) and requires `x + n` to have type `move_iterator<Iterator>` (which it always does). Consequently, this *Constraint* is always satisfied and it has no normative effect. The intent of the change in P0896R4 was that this operator be constrained to require addition on the base iterator to be well-formed and have type `Iterator`, which ensures that the other semantics of this operation are implementable.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4835.

1. Modify 23.5.3.9 [move.iter.nonmember] as indicated:

```
template<class Iterator>
  constexpr move_iterator<Iterator>
    operator+(iter_difference_t<Iterator> n, const move_iterator<Iterator>& x);
```

-2- *Constraints:* `x.base() + n` is well-formed and has type `Iterator`.

-3- *Returns:* `x + n`.

3361. `safe_range<SomeRange&>` case

Section: 24.4.2 [range.range] **Status:** Tentatively Ready **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2019-12-19 **Last modified:** 2021-09-20

Priority: 3

Discussion:

24.5.5 [range.dangling] p2 hints at how `safe_range` should allow lvalue ranges to model it. However, its wording doesn't take into account that case.

[2020-01 Priority set to 3 after review on the reflector.]

Previous resolution [SUPERSEDED]:

This wording is relative to N4842.

1. Modify 24.4.2 [range.range] as indicated:

```
template<class T>
  concept safe_range =
    range<T> &&
      (is_lvalue_reference_v<T> || enable_safe_range<remove_cvref_t<T>>);
```

-5- ~~Given an expression `E` such that `decltype((E))` is `T`,~~A type `T` models `safe_range` ~~only~~ if:

(5.1) — `is_lvalue_reference_v<T>` is `true`, or

(5.2) — given an expression `E` such that `decltype((E))` is `T`, ~~if~~ the validity of iterators obtained from the object denoted by `E` is not tied to the lifetime of that object.

-6- [*Note:* ~~Since the validity of iterators is not tied to the lifetime of an object whose type models `safe_range`, a~~A function can accept arguments of ~~such~~ a type that models `safe_range` ~~by value~~ and return iterators obtained from it without danger of dangling. — *end note*]

[2021-05-19 Tim updates wording]

The new wording below attempts to keep the "borrowed" property generally applicable to all models of `borrowed_range`, instead of bluntly carving out lvalue reference types.

[2021-09-20; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll in June.

Proposed resolution:

This wording is relative to N4885.

1. Modify 24.4.2 [range.range] as indicated:

```
template<class T>
  concept borrowed_range =
    range<T> &&
      (is_lvalue_reference_v<T> || enable_borrowed_range<remove_cvref_t<T>>);
```

-5- Let `U` be `remove_reference_t<T>` if `T` is an rvalue reference type, and `T` otherwise. Given ~~an expression `E` such that `decltype((E))` is `T`~~ a variable `u` of type `U`, `T` models `borrowed_range` only if the validity of iterators obtained from ~~the object denoted by `E`~~ `u` is not tied to the lifetime of that ~~object~~ variable.

-6- [*Note:* Since the validity of iterators is not tied to the lifetime of ~~an object~~a variable whose type models `borrowed_range`, a function ~~can accept arguments of~~ with a parameter of such a type ~~by value and~~ can return iterators obtained from it without danger of dangling. — *end note*]

3392. `ranges::distance()` cannot be used on a move-only iterator with a sized sentinel

Section: 23.4.4.3 [range.iter.op.distance] **Status:** Tentatively Ready **Submitter:** Patrick Palka **Opened:** 2020-02-07 **Last modified:** 2021-06-23

Priority: 3

Discussion:

One cannot use `ranges::distance(I, S)` to compute the distance between a move-only `counted_iterator` and the `default_sentinel`. In other words, the following is invalid

```
// iter is a counted_iterator with an move-only underlying iterator
ranges::distance(iter, default_sentinel);
```

and yet

```
(default_sentinel - iter);
```

is valid. The first example is invalid because `ranges::distance()` takes its first argument by value so when invoking it with an iterator lvalue argument the iterator must be copyable, which a move-only iterator is not. The second example is valid because `counted_iterator::operator-()` takes its iterator argument by `const` reference, so it doesn't require copyability of the `counted_iterator`.

This incongruency poses an inconvenience in generic code which uses `ranges::distance()` to efficiently compute the distance between two iterators or between an iterator-sentinel pair. Although it's a bit of an edge case, it would be good if `ranges::distance()` does the right thing when the iterator is a move-only lvalue with a sized sentinel.

If this is worth fixing, one solution might be to define a separate overload of `ranges::distance(I, S)` that takes its arguments by `const` reference, as follows.

[2020-02 Prioritized as P3 and LEWG Monday morning in Prague]

[2020-05-28; LEWG issue reviewing]

LEWG issue processing voted to accept the direction of 3392. Status change to Open.

```
Accept the direction of LWG3392

SF F N A SA
14 6 0 0 0
```

Previous resolution [SUPERSEDED]:

1. Modify 23.2 [iterator.synopsis], header `<iterator>` synopsis, as indicated:

```
#include <concepts>

namespace std {
  […]
  // 23.4.4 [range.iter.ops], range iterator operations
  namespace ranges {
    […]
    // 23.4.4.3 [range.iter.op.distance], ranges::distance
    template<input_or_output_iterator I, sentinel_for<I> S>
      requires (!sized_sentinel_for<S, I>)
        constexpr iter_difference_t<I> distance(I first, S last);
    template<input_or_output_iterator I, sized_sentinel_for<I> S>
      constexpr iter_difference_t<I> distance(const I& first, const S& last);
    template<range R>
      constexpr range_difference_t<R> distance(R&& r);
    […]
  }
  […]
}
```

2. Modify 23.4.4.3 [range.iter.op.distance] as indicated:

```
template<input_or_output_iterator I, sentinel_for<I> S>
  requires (!sized_sentinel_for<S, I>)
    constexpr iter_difference_t<I> ranges::distance(I first, S last);
```

-1- *Preconditions:* `[first, last)` denotes a range~~, or `[last, first)` denotes a range and `S` and `I` model `same_as<S, I> && sized_sentinel_for<S, I>`~~.

-2- *Effects:* ~~If `S` and `I` model `sized_sentinel_for<S, I>`, returns `(last - first)`; otherwise,~~ ~~r~~Returns the number of increments needed to get from `first` to `last`.

```
template<input_or_output_iterator I, sized_sentinel_for<I> S>
  constexpr iter_difference_t<I> ranges::distance(const I& first, const S& last);
```

-?- *Preconditions:* `S` and `I` model `sized_sentinel_for<S, I>` and either:

(?.1) — `[first, last)` denotes a range, or

(?.2) — `[last, first)` denotes a range and `S` and `I` model `same_as<S, I>`.

-? *Effects:* Returns `(last - first)`;

[2021-05-19 Tim updates wording]

The wording below removes the explicit precondition on the `sized_sentinel_for` overload of `distance`, relying instead on the semantic requirements of that concept and the "Effects: Equivalent to:" word of power. This also removes the potentially surprising inconsistency that given a non-empty `std::vector<int> v`, `ranges::distance(v.begin(), v.cend())` is well-defined but `ranges::distance(v.cend(), v.begin())` is currently undefined.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 23.2 [iterator.synopsis], header `<iterator>` synopsis, as indicated:

```
[…]

namespace std {
  […]
  // 23.4.4 [range.iter.ops], range iterator operations
  namespace ranges {
    […]
    // 23.4.4.3 [range.iter.op.distance], ranges::distance
    template<input_or_output_iterator I, sentinel_for<I> S>
      requires (!sized_sentinel_for<S, I>)
        constexpr iter_difference_t<I> distance(I first, S last);
    template<input_or_output_iterator I, sized_sentinel_for<I> S>
      constexpr iter_difference_t<I> distance(const I& first, const S& last);
    template<range R>
      constexpr range_difference_t<R> distance(R&& r);
    […]
  }
  […]
}
```

2. Modify 23.4.4.3 [range.iter.op.distance] as indicated:

```
template<input_or_output_iterator I, sentinel_for<I> S>
  requires (!sized_sentinel_for<S, I>)
    constexpr iter_difference_t<I> ranges::distance(I first, S last);
```

-1- *Preconditions:* `[first, last)` denotes a range~~, or `[last, first)` denotes a range and `S` and `I` model `same_as<S, I> && sized_sentinel_for<S, I>`~~.

-2- ~~*Effects:* If `S` and `I` model `sized_sentinel_for<S, I>`, returns `(last - first)`; otherwise, returns the~~ *Returns:* The number of increments needed to get from `first` to `last`.

```
template<input_or_output_iterator I, sized_sentinel_for<I> S>
  constexpr iter_difference_t<I> ranges::distance(const I& first, const S& last);
```

-?- *Effects:* Equivalent to `return last - first;`

3407. Some problems with the wording changes of P1739R4

Section: 24.7.8.1 [range.take.overview], 24.6.4 [range.iota] **Status:** Tentatively Ready **Submitter:** Patrick Palka **Opened:** 2020-02-21 **Last modified:** 2021-06-19

Priority: 2

Discussion:

Section 6.1 of P1739R4 changes the specification of `views::take` as follows:

> -2- The name `views::take` denotes a range adaptor object (24.7.2 [range.adaptor.object]). ~~Given subexpressions `E` and `F`, the expression `views::take(E, F)` is expression-equivalent to `take_view{E, F}`.~~ Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::take(E, F)` is ill-formed. Otherwise, the expression `views::take(E, F)` is expression-equivalent to:
>
> — if `T` is a specialization of `ranges::empty_view` (24.6.2.2 [range.empty.view]), then `((void) F, decay-copy(E))`;
>
> — otherwise, if `T` models `random_access_range` and `sized_range` and is
>
> — a specialization of `span` (22.7.3 [views.span]) where `T::extent == dynamic_extent`,
>
> — a specialization of `basic_string_view` (21.4 [string.view]),
>
> — a specialization of `ranges::iota_view` (24.6.4.2 [range.iota.view]), or
>
> — a specialization of `ranges::subrange` (24.5.4 [range.subrange]),
>
> then `T{ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F)}`, except that `E` is evaluated only once;
>
> — otherwise, `ranges::take_view{E, F}`.

Consider the case when `T = subrange<counted_iterator<int>, default_sentinel_t>`. Then according to the above wording, `views::take(E, F)` is expression-equivalent to

```
T{ranges::begin(E), ranges:begin(E) + min<D>(ranges::size(E), F)};   (*)
```

But this expression is ill-formed for the `T` we chose because `subrange<counted_iterator<int>, default_sentinel_t>` has no matching constructor that takes an `iterator-iterator` pair.

More generally the above issue applies anytime `T` is a specialization of `subrange` that does not model `common_range`. But a similar issue also exists when `T` is a specialization of `iota_view` whose value type differs from its bound type. In this case yet another issue arises: In order for the expression `(*)` to be well-formed when `T` is a specialization of `iota_view`, we need to be able to construct an `iota_view` out of an `iterator-iterator` pair, and for that it seems we need to add another constructor to `iota_view`.

[2020-02-24, Casey comments]

Furthermore, the pertinent subrange constructor is only available when `subrange::StoreSize` is `false` (i.e., when either the `subrange` specialization's third template argument is not `subrange_kind::sized` or its iterator and sentinel types `I` and `S` model `sized_sentinel_for<S, I>`).

[2020-03-16, Tomasz comments]

A similar problem occurs for the `views::drop` for the `subrange<I, S, subrange_kind::sized>`, that explicitly stores size (i.e. `sized_sentinel_for<I, S>` is `false`). In such case, the `(iterator, sentinel)` constructor that `views::drop` will be expression-equivalent is not available.

[2020-03-29 Issue Prioritization]

Priority to 2 after reflector discussion.

[2021-05-18 Tim adds wording]

The proposed resolution below is based on the MSVC implementation, with one caveat: the MSVC implementation uses a SCARY iterator type for `iota_view` and therefore its `iota_view` case for `take` is able to directly construct the new view from the iterator type of the original. This is not required to work, so the wording below constructs the `iota_view` from the result of dereferencing the iterators instead in this case.

[2021-06-18 Tim syncs wording to the current working draft]

The wording below also corrects the size calculation in the presence of integer-class types.

[2021-08-20; LWG telecon]

Set status to Tentatively Ready after telecon review.

Proposed resolution:

This wording is relative to N4892.

1. Edit 24.7.8.1 [range.take.overview] as indicated:

> -2- The name `views::take` denotes a range adaptor object (24.7.2 [range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::take(E, F)` is ill-formed. Otherwise, the expression `views::take(E, F)` is expression-equivalent to:
>
> (2.1) — if `T` is a specialization of `ranges::empty_view` (24.6.2.2 [range.empty.view]), then `((void) F, decay-copy(E))`, except that the evaluations of `E` and `F` are indeterminately sequenced;
>
> (2.2) — otherwise, if `T` models `random_access_range` and `sized_range`, and is a specialization of `span` (22.7.3 [views.span]), `basic_string_view` (21.4 [string.view]), or `ranges::subrange` (24.5.4 [range.subrange]), then `U(ranges::begin(E), ranges::begin(E) + std::min<D>(ranges::distance(E), F))`, except that `E` is evaluated only once, where `U` is a type determined as follows:
>
> (2.2.1) — if `T` is a specialization of `span` ~~(22.7.3 [views.span]) where `T::extent == dynamic_extent`~~, then `U` is `span<typename T::element_type>`;
>
> (2.2.2) — otherwise, if `T` is a specialization of `basic_string_view` ~~(21.4 [string.view])~~, then `U` is `T`;
>
> ~~(2.2.3) — a specialization of `ranges::iota_view` (24.6.4.2 [range.iota.view]), or~~
>
> (2.2.4) — otherwise, `T` is a specialization of `ranges::subrange` ~~(24.5.4 [range.subrange])~~, and `U` is `ranges::subrange<iterator_t<T>>`;
>
> ~~then `T{ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F)}`, except that `E` is evaluated only once;~~
>
> (2.?) — otherwise, if `T` is a specialization of `ranges::iota_view` (24.6.4.2 [range.iota.view]) that models `random access range` and `sized range`, then `ranges::iota view(*ranges::begin(E), *(ranges::begin(E) + std::min<D>(ranges::distance(E), F)))`, except that `E` and `F` are each evaluated only once;
>
> (2.3) — otherwise, `ranges::take_view(E, F)`.

2. Edit 24.7.10.1 [range.drop.overview] as indicated:

> -2- The name `views::drop` denotes a range adaptor object (24.7.2 [range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model

convertible_to<D>, views::drop(E, F) is ill-formed. Otherwise, the expression views::drop(E, F) is expression-equivalent to:

(2.1) — if T is a specialization of ranges::empty_view (24.6.2.2 [range.empty.view]), then ((void) F, *decay-copy*(E)), except that the evaluations of E and F are indeterminately sequenced;

(2.2) — otherwise, if T models random_access_range and sized_range, and is

(2.2.1) —a specialization of span (22.7.3 [views.span]) ~~where T::extent == dynamic_extent~~,

(2.2.2) — a specialization of basic_string_view (21.4 [string.view]),

(2.2.3) — a specialization of ranges::iota_view (24.6.4.2 [range.iota.view]), or

(2.2.4) — a specialization of ranges::subrange (24.5.4 [range.subrange]) where T::*StoreSize* is false,

then ~~T~~U(ranges::begin(E) + std::min<D>(ranges::~~size~~distance(E), F), ranges::end(E)), except that E is evaluated only once, where U is span<typename T::element_type> if T is a specialization of span and T otherwise;

(2.?) — otherwise, if T is a specialization of ranges::subrange (24.5.4 [range.subrange]) that models random_access_range and sized_range, then T(ranges::begin(E) + std::min<D>(ranges::distance(E), F), ranges::end(E), *to-unsigned-like*(ranges::distance(E) - std::min<D>(ranges::distance(E), F))), except that E and F are each evaluated only once;

(2.3) — otherwise, ranges::drop_view(E, F).

3422. Issues of seed_seq's constructors

Section: 26.6.8.1 [rand.util.seedseq] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2020-03-25 **Last modified:** 2021-06-23

Priority: 3

View other active issues in [rand.util.seedseq].

View all other issues in [rand.util.seedseq].

Discussion:

26.6.8.1 [rand.util.seedseq] says that std::seed_seq has following 3 constructors:

#1: seed_seq()

#2: template<class T> seed_seq(initializer_list<T> il)

#3: template<class InputIterator> seed_seq(InputIterator begin, InputIterator end)

The default constructor (#1) has no precondition and does not throw, and vector<result_type>'s default constructor is already noexcept since C++17, so #1 should also be noexcept.

Despite that the vector<result_type> member is exposition-only, current implementations (at least libc++, libstdc++ and MSVC STL) all hold it as the only data member of seed_seq, even with different names. And #1 is already noexcept in libc++ and libstdc++.

These constructors are not constrained, so #3 would never be matched in list-initialization. Consider following code:

```
#include <random>
#include <vector>

int main()
{
  std::vector<int> v(32);
  std::seed_seq{std::begin(v), std::end(v)}; // error: #2 matched and T is not an integer type
  std::seed_seq(std::begin(v), std::end(v)); // OK
}
```

#3 should be made available in list-initialization by changing *Mandates* in 26.6.8.1 [rand.util.seedseq]/3 to *Constraints* IMO.

[2020-04-18 Issue Prioritization]

Priority to 3 after reflector discussion.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4861.

1. Modify 26.6.8.1 [rand.util.seedseq] as indicated:

```
class seed_seq {
public:
  // types
  using result_type = uint_least32_t;

  // constructors
  seed_seq() noexcept;
  […]
};

seed_seq() noexcept;
```

-1- *Postconditions:* `v.empty()` is `true`.

~~-2- *Throws:* Nothing.~~

```
template<class T>
  seed_seq(initializer_list<T> il);
```

-3- *Constraints*~~*Mandates*~~: `T` is an integer type.

-4- *Effects:* Same as `seed_seq(il.begin(), il.end())`.

3470. `convertible-to-non-slicing` seems to reject valid case

Section: 24.5.4 [range.subrange] **Status:** Tentatively Ready **Submitter:** S. B. Tam **Opened:** 2020-07-26 **Last modified:** 2021-06-23

Priority: 3

View other active issues in [range.subrange].

View all other issues in [range.subrange].

Discussion:

Consider

```
#include <ranges>

int main()
{
  int a[3] = { 1, 2, 3 };
  int* b[3] = { &a[2], &a[0], &a[1] };
  auto c = std::ranges::subrange<const int*const*>(b);
}
```

The construction of `c` is ill-formed because *`convertible-to-non-slicing`*`<int**, const int*const*>` is `false`, although the conversion does not involve object slicing.

I think `subrange` should allow such qualification conversion, just like `unique_ptr<T[]>` already does.

(Given that this constraint is useful in more than one context, maybe it deserves a named type trait?)

[2020-08-21; Reflector prioritization]

Set priority to 3 after reflector discussions.

[2021-05-19 Tim adds wording]

The wording below, which has been implemented and tested on top of libstdc++, uses the same technique we use for `unique_ptr`, `shared_ptr`, and `span`. It seems especially appropriate to have feature parity between `subrange` and `span` in this respect.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 24.5.4 [range.subrange] as indicated:

```
namespace std::ranges {
  template<class From, class To>
    concept uses-nonqualification-pointer-conversion = // exposition only
      is_pointer_v<From> && is_pointer_v<To> &&
      !convertible_to<remove_pointer_t<From>(*)[], remove_pointer_t<To>(*)[]>;

  template<class From, class To>
    concept convertible-to-non-slicing = // exposition only
      convertible_to<From, To> &&
      !uses-nonqualification-pointer-conversion<decay_t<From>, decay_t<To>>;
      !(is_pointer_v<decay_t<From>> &&
      is_pointer_v<decay_t<To>> &&
      not-same-as<remove_pointer_t<decay_t<From>>, remove_pointer_t<decay_t<To>>>
      );
  […]
}
[…]
```

3480. `directory_iterator` and `recursive_directory_iterator` are not C++20 ranges

Section: 29.12.11 [fs.class.directory.iterator], 29.12.12 [fs.class.rec.dir.itr] **Status:** Tentatively Ready **Submitter:** Barry Revzin **Opened:** 2020-08-27 **Last modified:** 2021-06-23

Priority: 3

View all other issues in [fs.class.directory.iterator].

Discussion:

`std::filesystem::directory_iterator` and `std::filesystem::recursive_directory_iterator` are intended to be ranges, but both fail to satisfy the concept `std::ranges::range`.

They both opt in to being a range the same way, via non-member functions:

```
directory_iterator begin(directory_iterator iter) noexcept;
directory_iterator end(const directory_iterator&) noexcept;

recursive_directory_iterator begin(recursive_directory_iterator iter) noexcept;
recursive_directory_iterator end(const recursive_directory_iterator&) noexcept;
```

This is good enough for a range-based for statement, but for the `range` concept, non-member `end` is looked up in a context that includes (24.3.3 [range.access.end]/2.6) the declarations:

```
void end(auto&) = delete;
void end(const auto&) = delete;
```

Which means that non-`const` `directory_iterator` and non-`const` `recursive_directory_iterator`, the `void end(auto&)` overload ends up being a better match and thus the CPO `ranges::end` doesn't find a candidate. Which means that `{recursive_,}directory_iterator` is not a range, even though `const {recursive_,}directory_iterator` *is* a range.

This could be fixed by having the non-member `end` for both of these types just take by value (as libstdc++ currently does anyway) or by adding member functions `begin() const` and `end() const`.

A broader direction would be to consider removing the poison pill overloads. Their motivation from P0970 was to support what are now called borrowed ranges — but that design now is based on specializing a variable template instead of providing a non-member `begin` that takes an rvalue, so the initial motivation simply no longer exists. And, in this particular case, causes harm.

[2020-09-06; Reflector prioritization]

Set priority to 3 during reflector discussions.

[2021-02-22, Barry Revzin comments]

When we do make whichever of the alternative adjustments necessary such that `range<directory_iterator>` is `true`, we should also remember to specialize `enable_borrowed_range` for both types to be `true` (since the iterator is the range, this is kind of trivially true).

[2021-05-17, Tim provides wording]

Both MSVC and libstdc++'s `end` already take its argument by value, so the wording below just does that. Any discussion about changing or removing the poison pills is probably better suited for a paper.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Edit 29.12.4 [fs.filesystem.syn], header `<filesystem>` synopsis, as indicated:

```
[…]
namespace std::filesystem {
  […]

  // 29.12.11.3 [fs.dir.itr.nonmembers], range access for directory iterators
  directory_iterator begin(directory_iterator iter) noexcept;
  directory_iterator end(const directory_iterator&) noexcept;

  […]

  // 29.12.12.3 [fs.rec.dir.itr.nonmembers], range access for recursive directory iterators
  recursive_directory_iterator begin(recursive_directory_iterator iter) noexcept;
  recursive_directory_iterator end(const recursive_directory_iterator&) noexcept;

  […]
}


namespace std::ranges {
  template<>
  inline constexpr bool enable_borrowed_range<filesystem::directory_iterator> = true;
  template<>
  inline constexpr bool enable_borrowed_range<filesystem::recursive_directory_iterator> = true;

  template<>
  inline constexpr bool enable_view<filesystem::directory_iterator> = true;
  template<>
  inline constexpr bool enable_view<filesystem::recursive_directory_iterator> = true;
}
```

2. Edit 29.12.11.3 [fs.dir.itr.nonmembers] as indicated:

-1- These functions enable range access for `directory_iterator`.

```
directory_iterator begin(directory_iterator iter) noexcept;
```

-2- *Returns*: `iter`.

```
directory_iterator end(const directory_iterator&) noexcept;
```

-3- *Returns*: `directory_iterator()`.

3. Edit 29.12.12.3 [fs.rec.dir.itr.nonmembers] as indicated:

-1- These functions enable use of `recursive_directory_iterator` with range-based for statements.

```
recursive_directory_iterator begin(recursive_directory_iterator iter) noexcept;
```

-2- *Returns*: `iter`.

```
recursive_directory_iterator end(const recursive_directory_iterator&) noexcept;
```

-3- *Returns*: `recursive_directory_iterator()`.

3498. Inconsistent `noexcept`-specifiers for `basic_syncbuf`

Section: 29.11.2.1 [syncstream.syncbuf.overview], 29.11.2.3 [syncstream.syncbuf.assign] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2020-11-10 **Last modified:** 2021-06-23

Priority: 3

View all other issues in [syncstream.syncbuf.overview].

Discussion:

The synopsis in 29.11.2.1 [syncstream.syncbuf.overview] shows the move assignment operator and `swap` member as potentially throwing. The detailed descriptions in 29.11.2.3 [syncstream.syncbuf.assign] are `noexcept`.

Daniel:

This mismatch is already present in the originally accepted paper P0053R7, so this is nothing that could be resolved editorially.

[2020-11-21; Reflector prioritization]

Set priority to 3 during reflector discussions.

[2021-05-22 Tim adds PR]

The move assignment is specified to call `emit()` which can throw, and there's nothing in the wording providing for catching/ignoring the exception, so it can't be `noexcept`. The `swap` needs to call `basic_streambuf::swap`, which isn't `noexcept`, so it shouldn't be `noexcept` either.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 29.11.2.3 [syncstream.syncbuf.assign] as indicated:

```
basic_syncbuf& operator=(basic_syncbuf&& rhs) noexcept;
```

-1- *Effects*: […]

-2- *Postconditions*: […]

-3- *Returns*: […]

-4- *Remarks*: […]

```
void swap(basic_syncbuf& other) noexcept;
```

-5- *Preconditions*: […]

-6- *Effects*: […]

3535. `join_view::iterator::iterator_category` and `::iterator_concept` lie

Section: 24.7.12.3 [range.join.iterator] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2021-03-16 **Last modified:** 2021-06-23

Priority: 2

View other active issues in [range.join.iterator].

View all other issues in [range.join.iterator].

Discussion:

Per 24.7.12.3 [range.join.iterator]/1, `join_view::iterator::iterator_concept` denotes `bidirectional_iterator_tag` if *`ref-is-glvalue`* is `true` and *`Base`* and `range_reference_t<Base>` each model `bidirectional_range`. Similarly, paragraph 2 says that `join_view::iterator::iterator_category` is present if *`ref-is-glvalue`* is `true`, and denotes `bidirectional_iterator_tag` if the categories of the iterators of both *`Base`* and `range_reference_t<Base>` derive from `bidirectional_iterator_tag`.

However, the constraints on the declarations of `operator--` and `operator--(int)` in the synopsis that immediately precedes paragraph 1 disagree. Certainly they also consistently require *`ref-is-glvalue`* `&&` `bidirectional_range<Base>` `&&` `bidirectional_range<range_reference_t<Base>>`, but they additionally require `common_range<range_reference_t<Base>>`. So as currently specified, this iterator sometimes declares itself to be bidirectional despite not implementing `--`. This is not incorrect for iterator_concept — recall that `iterator_concept` is effectively an upper bound since the concepts require substantial syntax — but slightly misleading. It is, however, very much incorrect for `iterator_category` which must not denote a type derived from a tag that corresponds to a stronger category than that iterator implements.

It's worth pointing out, that LWG 3313 fixed the constraints on `operator--()` and `operator--(int)` by adding the `common_range` requirements, but failed to make a consistent change to the definitions of `iterator_concept` and `iterator_category`.

[2021-04-04; Daniel comments]

The below proposed wording can be compared as being based on N4885.

[2021-04-20; Reflector poll]

Priority set to 2.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to the post-2021-February-virtual-meeting working draft.

1. Modify 24.7.12.3 [range.join.iterator] as indicated:

-1- *iterator*::iterator_concept is defined as follows:

(1.1) — If *ref-is-glvalue* is true, ~~and~~ *Base* ~~and range_reference_t<Base> each~~ models bidirectional_range, and range_reference_t<*Base*> models both bidirectional_range and common_range, then iterator_concept denotes bidirectional_iterator_tag.

[…]

[…]

-2- The member *typedef-name* iterator_category is defined if and only if *ref-is-glvalue* is true, *Base* models forward_range, and range_reference_t<*Base*> models forward_range. In that case, *iterator*::iterator_category is defined as follows:

(2.1) — Let *OUTERC* denote iterator_traits<iterator_t<*Base*>>::iterator_category, and let *INNERC* denote iterator_traits<iterator_t<range_reference_t<*Base*>>>::iterator_category.

(2.2) — If *OUTERC* and *INNERC* each model derived_from<bidirectional_iterator_tag>, and range_reference_t<*Base*> models common_range, iterator_category denotes bidirectional_iterator_tag.

[…]

3554. `chrono::parse` needs `const charT*` and `basic_string_view<charT>` overloads

Section: 27.13 [time.parse] **Status:** Tentatively Ready **Submitter:** Howard Hinnant **Opened:** 2021-05-22 **Last modified:** 2021-06-23

Priority: 2

View all other issues in [time.parse].

Discussion:

The chrono::parse functions take const basic_string<charT, traits, Alloc>& parameters to specify the format strings for the parse. Due to an oversight on my part in the proposal, overloads taking const charT* and basic_string_view<charT, traits> were omitted. These are necessary when the supplied arguments is a string literal or string_view respectively:

```
in >> parse("%F %T", tp);
```

These overloads have been implemented in the example implementation.

[2021-05-26; Reflector poll]

Set priority to 2 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to N4885.

1. Modify 27.2 [time.syn], header <chrono> synopsis, as indicated:

```
[…]
namespace chrono {
  // 27.13 [time.parse], parsing
  template<class charT, class Parsable>
    unspecified
      parse(const charT* fmt, Parsable& tp);

  template<class charT, class traits, class Parsable>
    unspecified
      parse(basic_string_view<charT, traits> fmt, Parsable& tp);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const charT* fmt, Parsable& tp,
            basic_string<charT, traits, Alloc>& abbrev);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(basic_string_view<charT, traits> fmt, Parsable& tp,
            basic_string<charT, traits, Alloc>& abbrev);
```

```
  template<class charT, class Parsable>
    unspecified
      parse(const charT* fmt, Parsable& tp, minutes& offset);

  template<class charT, class traits, class Parsable>
    unspecified
      parse(basic_string_view<charT, traits> fmt, Parsable& tp, minutes& offset);

  template<class charT, class traits, class Alloc, class Parsable>
  unspecified
    parse(const charT* fmt, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev, minutes& offset);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(basic_string_view<charT, traits> fmt, Parsable& tp,
            basic_string<charT, traits, Alloc>& abbrev, minutes& offset);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const basic_string<charT, traits, Alloc>& format, Parsable& tp);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const basic_string<charT, traits, Alloc>& format, Parsable& tp,
            basic_string<charT, traits, Alloc>& abbrev);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const basic_string<charT, traits, Alloc>& format, Parsable& tp,
            minutes& offset);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const basic_string<charT, traits, Alloc>& format, Parsable& tp,
            basic_string<charT, traits, Alloc>& abbrev, minutes& offset);

  […]
}
[…]
```

2. Modify 27.13 [time.parse] as indicated:

```
template<class charT, class Parsable>
  unspecified
    parse(const charT* fmt, Parsable& tp);
```

-?- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT>&>(), fmt, tp)
```

is well-formed when treated as an unevaluated operand.

-?- *Effects:* Equivalent to `return parse(basic_string<charT>{fmt}, tp);`

```
template<class charT, class traits, class Parsable>
  unspecified
    parse(basic_string_view<charT, traits> fmt, Parsable& tp);
```

-?- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.data(), tp)
```

is well-formed when treated as an unevaluated operand.

-?- *Effects:* Equivalent to `return parse(basic_string<charT, traits>{fmt}, tp);`

```
template<class charT, class traits, class Alloc, class Parsable>
  unspecified
    parse(const charT* fmt, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev);
```

-?- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt, tp, addressof(abbrev))
```

is well-formed when treated as an unevaluated operand.

-?- *Effects:* Equivalent to `return parse(basic_string<charT, traits, Alloc>{fmt}, tp, abbrev);`

```
template<class charT, class traits, class Alloc, class Parsable>
  unspecified
```

```
  parse(basic_string_view<charT, traits> fmt, Parsable& tp,
        basic_string<charT, traits, Alloc>& abbrev);
```

-?- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.data(), tp, addressof(abbrev))
```

is well-formed when treated as an unevaluated operand.

-?- *Effects:* Equivalent to `return parse(basic_string<charT, traits, Alloc>{fmt}, tp, abbrev);`

```
template<class charT, class Parsable>
  unspecified
  parse(const charT* fmt, Parsable& tp, minutes& offset);
```

-?- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT>&>(), fmt, tp,
            declval<basic_string<charT>*>(),
            &offset)
```

is well-formed when treated as an unevaluated operand.

-?- *Effects:* Equivalent to `return parse(basic_string<charT>{fmt}, tp, offset);`

```
template<class charT, class traits, class Parsable>
  unspecified
  parse(basic_string_view<charT, traits> fmt, Parsable& tp, minutes& offset);
```

-?- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.data(), tp,
            declval<basic_string<charT, traits>*>(),
            &offset)
```

is well-formed when treated as an unevaluated operand.

-?- *Effects:* Equivalent to `return parse(basic_string<charT, traits>{fmt}, tp, offset);`

```
template<class charT, class traits, class Alloc, class Parsable>
unspecified
  parse(const charT* fmt, Parsable& tp,
        basic_string<charT, traits, Alloc>& abbrev, minutes& offset);
```

-?- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt, tp, addressof(abbrev), &offset)
```

is well-formed when treated as an unevaluated operand.

-?- *Effects:* Equivalent to `return parse(basic_string<charT, traits, Alloc>{fmt}, tp, abbrev, offset);`

```
template<class charT, class traits, class Alloc, class Parsable>
  unspecified
  parse(basic_string_view<charT, traits> fmt, Parsable& tp,
        basic_string<charT, traits, Alloc>& abbrev, minutes& offset);
```

-?- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.data(), tp, addressof(abbrev), &offset)
```

is well-formed when treated as an unevaluated operand.

-?- *Effects:* Equivalent to `return parse(basic_string<charT, traits, Alloc>{fmt}, tp, abbrev, offset);`

```
template<class charT, class traits, class Alloc, class Parsable>
  unspecified
    parse(const basic_string<charT, traits, Alloc>& format, Parsable& tp);
```

-2- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp)
```

is well-formed when treated as an unevaluated operand.

-3- *Returns:* A manipulator such that the expression `is >> parse(fmt, tp)` has type `I`, has value `is`, and calls `from_stream(is, fmt.c_str(), tp)`.

[2021-06-02 Tim comments and provides updated wording]

The current specification suggests that `parse` takes a reference to the format string (stream extraction on the resulting manipulator is specified to call `from_stream` on `fmt.c_str()`, not some copy), so we can't call it with a temporary string.

Additionally, since the underlying API (`from_stream`) requires a null-terminated string, the usual practice is to avoid providing a `string_view` overload (see, e.g., LWG 3430). The wording below therefore only provides `const charT*` overloads. It does not use "Equivalent to" to avoid requiring that the `basic_string` overload and the `const charT*` overload return the same type. As the manipulator is intended to be immediately consumed, the wording also adds normative encouragement to make misuse more difficult.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 27.2 [time.syn], header <chrono> synopsis, as indicated:

```
[…]
namespace chrono {
  // 27.13 [time.parse], parsing
  template<class charT, class Parsable>
    unspecified
      parse(const charT* fmt, Parsable& tp);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const basic_string<charT, traits, Alloc>& formatfmt, Parsable& tp);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const charT* fmt, Parsable& tp,
            basic_string<charT, traits, Alloc>& abbrev);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const basic_string<charT, traits, Alloc>& formatfmt, Parsable& tp,
            basic_string<charT, traits, Alloc>& abbrev);

  template<class charT, class Parsable>
    unspecified
      parse(const charT* fmt, Parsable& tp, minutes& offset);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const basic_string<charT, traits, Alloc>& formatfmt, Parsable& tp,
            minutes& offset);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const charT* fmt, Parsable& tp,
            basic_string<charT, traits, Alloc>& abbrev, minutes& offset);

  template<class charT, class traits, class Alloc, class Parsable>
    unspecified
      parse(const basic_string<charT, traits, Alloc>& formatfmt, Parsable& tp,
            basic_string<charT, traits, Alloc>& abbrev, minutes& offset);

  […]
}
[…]
```

2. Modify 27.13 [time.parse] as indicated:

-1- Each `parse` overload specified in this subclause calls `from_stream` unqualified, so as to enable argument dependent lookup (6.5.4 [basic.lookup.argdep]). In the following paragraphs, let `is` denote an object of type `basic_istream<charT, traits>` and let `I` be `basic_istream<charT, traits>&`, where `charT` and `traits` are template parameters in that context.

-?- *Recommended practice:* Implementations should make it difficult to accidentally store or use a manipulator that may contain a dangling reference to a format string, for example by making the manipulators produced by `parse` immovable and preventing stream extraction into an lvalue of such a manipulator type.

```
template<class charT, class Parsable>
  unspecified
    parse(const charT* fmt, Parsable& tp);
template<class charT, class traits, class Alloc, class Parsable>
  unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp);
```

-?- Let *F* be `fmt` for the first overload and `fmt.c_str()` for the second overload. Let `traits` be `char_traits<charT>` for the first overload.

-2- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str()F, tp)
```

is well-formed when treated as an unevaluated operand.

-3- *Returns:* A manipulator such that the expression `is >> parse(fmt, tp)` has type `I`, has value `is`, and calls `from_stream(is, fmt.c_str()F, tp)`.

```
template<class charT, class traits, class Alloc, class Parsable>
  unspecified
    parse(const charT* fmt, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev);
template<class charT, class traits, class Alloc, class Parsable>
  unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev);
```

-?- Let *F* be `fmt` for the first overload and `fmt.c_str()` for the second overload.

-4- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str()F, tp, addressof(abbrev))
```

is well-formed when treated as an unevaluated operand.

-5- *Returns:* A manipulator such that the expression `is >> parse(fmt, tp, abbrev)` has type `I`, has value `is`, and calls `from_stream(is, fmt.c_str()F, tp, addressof(abbrev))`.

```
template<class charT, class Parsable>
  unspecified
    parse(const charT* fmt, Parsable& tp, minutes& offset);

template<class charT, class traits, class Alloc, class Parsable>
  unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
          minutes& offset);
```

-?- Let *F* be `fmt` for the first overload and `fmt.c_str()` for the second overload. Let `traits` be `char_traits<charT>` and `Alloc` be `allocator<charT>` for the first overload.

-6- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(),
            fmt.c_str()F, tp,
            declval<basic_string<charT, traits, Alloc>*>(),
            &offset)
```

is well-formed when treated as an unevaluated operand.

-7- *Returns:* A manipulator such that the expression `is >> parse(fmt, tp, offset)` has type `I`, has value `is`, and calls:

```
from_stream(is,
            fmt.c_str()F, tp,
            static_cast<basic_string<charT, traits, Alloc>*>(nullptr),
            &offset)
```

```
template<class charT, class traits, class Alloc, class Parsable>
  unspecified
    parse(const charT* fmt, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev, minutes& offset);

template<class charT, class traits, class Alloc, class Parsable>
  unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev, minutes& offset);
```

-?- Let *F* be `fmt` for the first overload and `fmt.c_str()` for the second overload.

-8- *Constraints:* The expression

```
from_stream(declval<basic_istream<charT, traits>&>(),
            fmt.c_str()F, tp, addressof(abbrev), &offset)
```

is well-formed when treated as an unevaluated operand.

-9- *Returns:* A manipulator such that the expression `is >> parse(fmt, tp, abbrev, offset)` has type `I`, has value `is`, and calls `from_stream(is, ~~fmt.c_str()~~F, tp, addressof(abbrev), &offset)`.

3557. The `static_cast` expression in `convertible_to` has the wrong operand

Section: 18.4.4 [concept.convertible] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-05-26 **Last modified:** 2021-06-07

Priority: Not Prioritized

View other active issues in [concept.convertible].

View all other issues in [concept.convertible].

Discussion:

The specification of `convertible_to` implicitly requires `static_cast<To>(f())` to be equality-preserving. Under 18.2 [concepts.equality] p1, the operand of this expression is `f`, but what we really want is for `f()` to be treated as the operand. We should just use `declval` (which is treated specially by the definition of "operand" for this purpose) instead of reinventing the wheel.

[2021-06-07; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 18.4.4 [concept.convertible] as indicated:

```
template<class From, class To>
  concept convertible_to =
    is_convertible_v<From, To> &&
    requires~~(add_rvalue_reference_t<From> (&f)())~~ {
      static_cast<To>(~~f~~declval<From>());
    };
```

3559. Semantic requirements of `sized_range` is circular

Section: 24.4.3 [range.sized] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-06-03 **Last modified:** 2021-06-14

Priority: Not Prioritized

View all other issues in [range.sized].

Discussion:

24.4.3 [range.sized] p2.1 requires that for a `sized_range t`, `ranges::size(t)` is equal to `ranges::distance(t)`. But for a `sized_range`, `ranges::distance(t)` is simply `ranges::size(t)` cast to the difference type.

Presumably what we meant is that `distance(begin, end)` — the actual distance you get from traversing the range — is equal to what `ranges::size` produces, and not merely that casting from the size type to difference type is value-preserving. Otherwise, `sized_range` would be more or less meaningless.

[2021-06-14; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 24.4.3 [range.sized] as indicated:

```
template<class T>
  concept sized_range =
    range<T> &&
    requires(T& t) { ranges::size(t); };
```

-2- Given an lvalue `t` of type `remove_reference_t<T>`, `T` models `sized_range` only if

(2.1) — `ranges::size(t)` is amortized $\mathcal{O}(1)$, does not modify `t`, and is equal to `ranges::distance(ranges::begin(t), ranges::end(t)~~t~~)`, and

(2.2) — if `iterator_t<T>` models `forward_iterator`, `ranges::size(t)` is well-defined regardless of the evaluation of `ranges::begin(t)`.

3560. `ranges::equal` and `ranges::is_permutation` should short-circuit for `sized_ranges`

Section: 25.6.11 [alg.equal], 25.6.12 [alg.is.permutation] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-06-03 **Last modified:** 2021-06-23

Priority: Not Prioritized

View all other issues in [alg.equal].

Discussion:

`ranges::equal` and `ranges::is_permutation` are currently only required to short-circuit on different sizes if the iterator and sentinel of the two ranges pairwise model `sized_sentinel_for`. They should also short-circuit if the ranges are sized.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 25.6.11 [alg.equal] as indicated:

```
template<class InputIterator1, class InputIterator2>
  constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
                       InputIterator2 first2);
[…]
template<class InputIterator1, class InputIterator2,
         class BinaryPredicate>
  constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
                       InputIterator2 first2, InputIterator2 last2,
                       BinaryPredicate pred);
[…]
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
  requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
  constexpr bool ranges::equal(I1 first1, S1 last1, I2 first2, S2 last2,
                               Pred pred = {},
                               Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, class Pred = ranges::equal_to,
         class Proj1 = identity, class Proj2 = identity>
  requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
  constexpr bool ranges::equal(R1&& r1, R2&& r2, Pred pred = {},
                               Proj1 proj1 = {}, Proj2 proj2 = {});
```

[…]

-3- *Complexity:* If ~~the types of `first1`, `last1`, `first2`, and `last2`~~:

(3.1) — the types of `first1`, `last1`, `first2`, and `last2` meet the *Cpp17RandomAccessIterator* requirements (23.3.5.7 [random.access.iterators]) and `last1 - first1 != last2 - first2` for the overloads in namespace `std`;

(3.2) — the types of `first1`, `last1`, `first2`, and `last2` pairwise model `sized_sentinel_for` (23.3.4.8 [iterator.concept.sizedsentinel]) and `last1 - first1 != last2 - first2` for the first overload~~s~~ in namespace `ranges`,

(3.3) — `R1` and `R2` each model `sized_range` and `ranges::distance(r1) != ranges::distance(r2)` for the second overload in namespace `ranges`,

~~and `last1 - first1 != last2 - first2`,~~ then no applications of the corresponding predicate and each projection; otherwise, […]

2. Modify 25.6.12 [alg.is.permutation] as indicated:

```
template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
         sentinel_for<I2> S2, class Proj1 = identity, class Proj2 = identity,
         indirect_equivalence_relation<projected<I1, Proj1>,
                                       projected<I2, Proj2>> Pred = ranges::equal_to>
  constexpr bool ranges::is_permutation(I1 first1, S1 last1, I2 first2, S2 last2,
                                        Pred pred = {},
                                        Proj1 proj1 = {}, Proj2 proj2 = {});
template<forward_range R1, forward_range R2,
```

```
        class Proj1 = identity, class Proj2 = identity,
        indirect_equivalence_relation<projected<iterator_t<R1>, Proj1>,
                                      projected<iterator_t<R2>, Proj2>> Pred = ranges::equal_to>
constexpr bool ranges::is_permutation(R1&& r1, R2&& r2, Pred pred = {},
                                      Proj1 proj1 = {}, Proj2 proj2 = {});
```

[…]

-7- *Complexity:* No applications of the corresponding predicate and projections if:

(7.1) — for the first overload,

(7.1.1) — `S1` and `I1` model `sized_sentinel_for<S1, I1>`,

(7.1.2) — `S2` and `I2` model `sized_sentinel_for<S2, I2>`, and

(7.1.3) — `last1 - first1 != last2 - first2`~~;~~;

(7.2) — for the second overload, `R1` and `R2` each model `sized range`, and `ranges::distance(r1) != ranges::distance(r2)`.

Otherwise, exactly `last1 - first1` applications of the corresponding predicate and projections if `ranges::equal(first1, last1, first2, last2, pred, proj1, proj2)` would return `true`; otherwise, at worst $\mathcal{O}(N^2)$, where N has the value `last1 - first1`.

3561. Issue with internal counter in `discard_block_engine`

Section: 26.6.5.2 [rand.adapt.disc] **Status:** Tentatively Ready **Submitter:** Ilya Burylov **Opened:** 2021-06-03 **Last modified:** 2021-06-14

Priority: Not Prioritized

View all other issues in [rand.adapt.disc].

Discussion:

A `discard_block_engine` engine adaptor is described in 26.6.5.2 [rand.adapt.disc]. It produces random numbers from the underlying engine discarding `(p - r)` last values of `p - size` block, where `r` and `p` are template parameters of `std::size_t` type:

```
template<class Engine, size_t p, size_t r>
class discard_block_engine;
```

The transition algorithm for `discard_block_engine` is described as follows (paragraph 2):

> The transition algorithm discards all but r `> 0` values from each block of p `= r` values delivered by e. The state transition is performed as follows: If n `= r`, advance the state of `e` from e_i to e_{i+p-r} and set n to `0`. In any case, then increment n and advance `e`'s then-current state e_j to e_{j+1}.

Where n is of integer type. In the API of discard block engine, n is represented in the following way:

```
[…]
int n; // exposition only
```

In cases where `int` is equal to `int32_t`, overflow is possible for n that leads to undefined behavior. Such situation can happen when the `p` and `r` template parameters exceed `INT_MAX`.

This misleading exposition block leads to differences in implementations:

- GNU Libstdc++ uses `size_t` for `n` — in this case no overflow happened even if template parameters exceed `INT_MAX`
- LLVM Libcxx uses `int` for `n` together with a `static_assert` that is checking that `p` and `r` values are `<= INT_MAX`

Such difference in implementation makes code not portable and may potentially breaks random number sequence consistency between different implementors of C++ std lib.

The problematic use case is the following one:

```
#include <iostream>
#include <random>
#include <limits>

int main() {
  std::minstd_rand0 generator;

  constexpr std::size_t skipped_size = static_cast<std::size_t>(std::numeric_limits<int>::max());
  constexpr std::size_t block_size = skipped_size + 5;
  constexpr std::size_t used_block = skipped_size;
```

```
  std::cout << "block_size = " << block_size << std::endl;
  std::cout << "used_block = " << used_block << "\n" << std::endl;

  std::discard_block_engine<std::minstd_rand0, block_size, used_block> discard_generator;

  // Call discard procedures
  discard_generator.discard(used_block);
  generator.discard(block_size);

  // Call generation. Results should be equal
  for (std::int32_t i = 0; i < 10; ++i)
  {
    std::cout << discard_generator() << " should be equal " << generator() << std::endl;
  }
}
```

We see no solid reason for `n` to be an `int`, given that the relevant template parameters are `std::size_t`. It seems like a perfectly fine use case to generate random numbers in amounts larger than `INT_MAX`.

The proposal is to change exposition text to `std::size_t`:

```
size_t n; // exposition only
```

It will not mandate the existing libraries to break ABI, but at least guide for better implementation.

[2021-06-14; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 26.6.5.2 [rand.adapt.disc], class template `discard_block_engine` synopsis, as indicated:

```
    […]
    // generating functions
    result_type operator()();
    void discard(unsigned long long z);

    // property functions
    const Engine& base() const noexcept { return e; };

  private:
    Engine e; // exposition only
    int size_t n; // exposition only
  };
```

3563. `keys_view` example is broken

Section: 24.7.18.1 [range.elements.overview] **Status:** Tentatively Ready **Submitter:** Barry Revzin **Opened:** 2021-05-29 **Last modified:** 2021-09-20

Priority: 3

Discussion:

In 24.7.18.1 [range.elements.overview] we have:

> `keys_view` is an alias for `elements_view<views::all_t<R>, 0>`, and is useful for extracting keys from associative containers.
>
> [*Example 2:*
>
> ```
> auto names = keys_view{historical_figures};
> for (auto&& name : names) {
>   cout << name << ' ';  // prints Babbage Hamilton Lovelace Turing
> }
> ```
>
> — *end example*]

Where `keys_view` is defined as:

```
template<class R>
  using keys_view = elements_view<views::all_t<R>, 0>;
```

There's a similar example for `values_view`.

But class template argument deduction cannot work here, `keys_view(r)` cannot deduce `R` — it's a non-deduced context. At the very least, the example is wrong and should be rewritten as `views::keys(historical_figures)`. Or, I guess, as `keys_view<decltype(historical_figures)>(historical_figures)` (but really, not).

[2021-05-29 Tim comments and provides wording]

I have some ideas about making CTAD work for `keys_view` and `values_view`, but current implementations of alias template CTAD are not yet in a shape to permit those ideas to be tested. What *is* clear, though, is that the current definitions of `keys_view` and `values_view` can't possibly ever work for CTAD, because `R` is only used in a non-deduced context and so they can never meet the "deducible" constraint in 12.2.2.9 [over.match.class.deduct] p2.3.

The proposed resolution below removes the `views::all_t` transformation in those alias templates. This makes these aliases transparent enough to support CTAD out of the box when a view is used, and any questions about deduction guides on `elements_view` can be addressed later once the implementations become more mature. This also makes them consistent with all the other views: you can't write `elements_view<map<int, int>&, 0>` or `reverse_view<map<int, int>&>`, so why do we allow `keys_view<map<int, int>&>`? It is, however, a breaking change, so it is important that we get this in sooner rather than later.

The proposed resolution has been implemented and tested.

[2021-06-14; Reflector poll]

Set priority to 3 after reflector poll in August. Partially addressed by an editorial change.

Previous resolution [SUPERSEDED]:

This wording is relative to N4885.

1. Modify 24.2 [ranges.syn], header `<ranges>` synopsis, as indicated:

```
[…]
namespace std::ranges {
[…]

  // 24.7.18 [range.elements], elements view
  template<input_range V, size_t N>
    requires see below
  class elements_view;

  template<class T, size_t N>
    inline constexpr bool enable_borrowed_range<elements_view<T, N>> = enable_borrowed_range<T>;

  template<class R>
    using keys_view = elements_view<~~views::all_t<~~R~~>~~, 0>;
  template<class R>
    using values_view = elements_view<~~views::all_t<~~R~~>~~, 1>;

[…]
}
```

2. Modify 24.7.18.1 [range.elements.overview] as indicated:

-3- `keys_view` is an alias for `elements_view<~~views::all_t<~~R~~>~~, 0>`, and is useful for extracting keys from associative containers.

[*Example 2*:

```
auto names = keys_view{views::all(historical_figures)};
for (auto&& name : names) {
  cout << name << ' '; // prints Babbage Hamilton Lovelace Turing
}
```

— *end example*]

-4- `values_view` is an alias for `elements_view<~~views::all_t<~~R~~>~~, 1>`, and is useful for extracting values from associative containers.

[*Example 3*:

```
auto is_even = [](const auto x) { return x % 2 == 0; };
cout << ranges::count_if(values_view{views::all(historical_figures)}, is_even); // prints 2
```

— *end example*]

[2021-06-18 Tim syncs wording to latest working draft]

The examples have been editorially corrected, so the new wording simply changes the definition of `keys_view` and `values_view`.

[2021-09-20; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 24.2 [ranges.syn], header `<ranges>` synopsis, as indicated:

```
[…]
namespace std::ranges {
[…]

  // 24.7.18 [range.elements], elements view
  template<input_range V, size_t N>
    requires see below
  class elements_view;

  template<class T, size_t N>
    inline constexpr bool enable_borrowed_range<elements_view<T, N>> = enable_borrowed_range<T>;

  template<class R>
    using keys_view = elements_view<~~views::all_t<~~R~~>~~, 0>;
  template<class R>
    using values_view = elements_view<~~views::all_t<~~R~~>~~, 1>;

[…]
}
```

2. Modify 24.7.18.1 [range.elements.overview] as indicated:

-3- `keys_view` is an alias for `elements_view<`~~`views::all_t<`~~`R`~~`>`~~`, 0>`, and is useful for extracting keys from associative containers.

[*Example 2*: … — *end example*]

-4- `values_view` is an alias for `elements_view<`~~`views::all_t<`~~`R`~~`>`~~`, 1>`, and is useful for extracting values from associative containers.

[*Example 3*: … — *end example*]

3566. Constraint recursion for `operator<=>(optional<T>, U)`

Section: 20.6.8 [optional.comp.with.t] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2021-06-07 **Last modified:** 2021-06-23

Priority: Not Prioritized

View all other issues in [optional.comp.with.t].

Discussion:

The three-way-comparison operator for optionals and non-optionals is defined in [optional.comp.with.t] paragraph 25:

```
template<class T, three_way_comparable_with<T> U>
  constexpr compare_three_way_result_t<T, U>
    operator<=>(const optional<T>& x, const U& v);
```

-25- *Effects:* Equivalent to: `return bool(x) ? *x <=> v : strong_ordering::less;`

Checking `three_way_comparable_with` in particular requires checking that `x < v` is well-formed for an lvalue `const optional<T>` and lvalue `const U`. `x < v` can be rewritten as `(v <=> x) > 0`. If `U` is a specialization of `optional`, this overload could be used for `v <=> x`, but first we must check the constraints…

The straightforward fix for this recursion seems to be to refuse to check `three_way_comparable_with<T>` when `U` is a specialization of `optional`. MSVC has been shipping such a fix; our initial tests for this new operator triggered the recursion.

Previous resolution [SUPERSEDED]:

This wording is relative to N4885.

1. Modify 20.6.2 [optional.syn], header `<optional>` synopsis, as indicated:

```
[…]
// 20.6.8 [optional.comp.with.t], comparison with T
[…]
template<class T, class U> constexpr bool operator>=(const optional<T>&, const U&);
```

```
template<class T, class U> constexpr bool operator>=(const T&, const optional<U>&);
template<class T, three_way_comparable_with<T>class U>
  requires see below
    constexpr compare_three_way_result_t<T, U>
      operator<=>(const optional<T>&, const U&);

[…]
```

2. Modify 20.6.8 [optional.comp.with.t] as indicated:

```
template<class T, three_way_comparable_with<T>class U>
  requires (!is-specialization-of<U, optional>) && three_way_comparable_with<T, U>
    constexpr compare_three_way_result_t<T, U>
      operator<=>(const optional<T>&, const U&);
```

-?- The exposition-only trait template *is-specialization-of*<A, B> is a constant expression with value `true` when `A` denotes a specialization of the class template `B`, and `false` otherwise.

-25- *Effects:* Equivalent to: `return bool(x) ? *x <=> v : strong_ordering::less;`

[2021-06-14; Improved proposed wording based on reflector discussion]

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 20.6.2 [optional.syn], header <optional> synopsis, as indicated:

```
[…]
// 20.6.3 [optional.optional], class template optional
template<class T>
class optional;

template<class T>
  constexpr bool is-optional = false;               // exposition only
template<class T>
  constexpr bool is-optional<optional<T>> = true;   // exposition only

[…]
// 20.6.8 [optional.comp.with.t], comparison with T
[…]
template<class T, class U> constexpr bool operator>=(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator>=(const T&, const optional<U>&);
template<class T, three_way_comparable_with<T>class U>
  requires (!is-optional<U>) && three_way_comparable_with<T, U>
    constexpr compare_three_way_result_t<T, U>
      operator<=>(const optional<T>&, const U&);

[…]
```

2. Modify 20.6.8 [optional.comp.with.t] as indicated:

```
template<class T, three_way_comparable_with<T>class U>
  requires (!is-optional<U>) && three_way_comparable_with<T, U>
    constexpr compare_three_way_result_t<T, U>
      operator<=>(const optional<T>& x, const U& v);
```

-25- *Effects:* Equivalent to: `return bool(x) ? *x <=> v : strong_ordering::less;`

3567. Formatting move-only iterators take two

Section: 20.20.6.4 [format.context] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2021-06-08 **Last modified:** 2021-06-23

Priority: Not Prioritized

Discussion:

LWG 3539 fixed copies of potentially-move-only iterators in the `format_to` and `vformat_to` overloads, but missed the fact that member functions of `basic_format_context` are specified to copy iterators as well. In particular, 20.20.6.4 [format.context] states:

```
iterator out();
```

-7- *Returns:* `out_`.

```
void advance_to(iterator it);
```

-8- *Effects:* Equivalent to: `out_ = it;`

both of which appear to require copyability.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 20.20.6.4 [format.context] as indicated:

```
iterator out();
```

-7- ~~*Returns:* `out_`.~~*Effects:* Equivalent to: `return std::move(out_);`

```
void advance_to(iterator it);
```

-8- *Effects:* Equivalent to: `out_ = std::move(it);`

3568. `basic_istream_view` needs to initialize *`value_`*

Section: 24.6.5.2 [range.istream.view] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2021-06-15 **Last modified:** 2021-06-23

Priority: Not Prioritized

Discussion:

P2325R3 removes the default member initializers for `basic_istream_view`'s exposition-only *`stream_`* and *`value_`* members. Consequently, copying a `basic_istream_view` before the first call to `begin` may produce undefined behavior since doing so necessarily copies the uninitialized *`value_`*. We should restore *`value_`*'s default member initializer and remove this particular footgun.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

Wording relative to the post-202106 working draft (as near as possible). This PR is currently being implemented in MSVC.

1. Modify 24.6.5.2 [range.istream.view] as indicated:

```
namespace std::ranges {
  […]
  template<movable Val, class CharT, class Traits>
    requires default_initializable<Val> &&
             stream-extractable<Val, CharT, Traits>
  class basic_istream_view : public view_interface<basic_istream_view<Val, CharT, Traits>> {
    […]
    Val value_ = Val(); // exposition only
  };
}
```

3570. `basic_osyncstream::emit` should be an unformatted output function

Section: 29.11.3.3 [syncstream.osyncstream.members] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-06-19 **Last modified:** 2021-06-23

Priority: Not Prioritized

Discussion:

`basic_osyncstream::emit` seems rather similar to `basic_ostream::flush` — both are "flushing" operations that forward to member functions of the `streambuf` and set `badbit` if those functions fail. But the former isn't currently specified as an unformatted output function, so it a) doesn't construct or check a `sentry` and b) doesn't set `badbit` if `emit` exits via an exception. At least the latter appears to be clearly desirable — a `streambuf` operation that throws ordinarily results in `badbit` being set.

The reference implementation in P0053R7 constructs a `sentry` and only calls `emit` on the `streambuf` if the `sentry` converts to `true`, so this seems to be an accidental omission in the wording.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 29.11.3.3 [syncstream.osyncstream.members] as indicated:

```
void emit();
```

-1- *Effects:* Behaves as an unformatted output function (29.7.5.4 [ostream.unformatted]). After constructing a `sentry` object, c~~∈~~alls `sb.emit()`. If that call returns `false`, calls `setstate(ios_base::badbit)`.

3571. `flush_emit` should set `badbit` if the `emit` call fails

Section: 29.7.5.5 [ostream.manip] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-06-19 **Last modified:** 2021-06-23

Priority: Not Prioritized

View other active issues in [ostream.manip].

View all other issues in [ostream.manip].

Discussion:

`basic_osyncstream::emit` is specified to set `badbit` if it fails (29.11.3.3 [syncstream.osyncstream.members] p1), but the `emit` part of the `flush_emit` manipulator isn't, even though the `flush` part does set `badbit` if it fails.

More generally, given an `osyncstream s`, `s << flush_emit;` should probably have the same behavior as `s.flush(); s.emit()`.

The reference implementation linked in P0753R2 does set `badbit` on failure, so at least this part appears to be an oversight. As discussed in LWG 3570, `basic_osyncstream::emit` should probably be an unformatted output function, so the `emit` part of `flush_emit` should do so too.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 29.7.5.5 [ostream.manip] as indicated:

```
template<class charT, class traits>
  basic_ostream<charT, traits>& flush_emit(basic_ostream<charT, traits>& os);
```

-12- *Effects:* Calls `os.flush()`. Then, if `os.rdbuf()` is a `basic_syncbuf<charT, traits, Allocator>*`, called `buf` for the purpose of exposition, behaves as an unformatted output function (29.7.5.4 [ostream.unformatted]) of `os`. After constructing a `sentry` object, calls `buf->emit()`. If that call returns `false`, calls `os.setstate(ios_base::badbit)`.

3572. *`copyable-box`* should be fully `constexpr`

Section: 24.7.3 [range.copy.wrap] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-06-19 **Last modified:** 2021-07-17

Priority: Not Prioritized

Discussion:

P2231R1 made `optional` fully `constexpr`, but missed its cousin *`semiregular-box`* (which was renamed to *`copyable-box`* by P2325R3). Most operations of *`copyable-box`* are already `constexpr` simply because *`copyable-box`* is specified in terms of `optional`; the only missing ones are the assignment operators layered on top when the wrapped type isn't assignable itself.

[2021-06-23; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 24.7.3 [range.copy.wrap] as indicated:

-1- Many types in this subclause are specified in terms of an exposition-only class template *copyable-box*. *copyable-box*<T> behaves exactly like optional<T> with the following differences:

(1.1) — […]

(1.2) — […]

(1.3) — If copyable<T> is not modeled, the copy assignment operator is equivalent to:

```
constexpr copyable-box& operator=(const copyable-box& that)
  noexcept(is_nothrow_copy_constructible_v<T>) {
  if (this != addressof(that)) {
    if (that) emplace(*that);
    else reset();
  }
  return *this;
}
```

(1.4) — If movable<T> is not modeled, the move assignment operator is equivalent to:

```
constexpr copyable-box& operator=(copyable-box&& that)
  noexcept(is_nothrow_move_constructible_v<T>) {
  if (this != addressof(that)) {
    if (that) emplace(std::move(*that));
    else reset();
  }
  return *this;
}
```

3573. Missing *Throws* element for `basic_string_view(It begin, End end)`

Section: 21.4.3.2 [string.view.cons] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2021-07-13 **Last modified:** 2021-07-17

Priority: Not Prioritized

View other active issues in [string.view.cons].

View all other issues in [string.view.cons].

Discussion:

The standard does not specify the exceptions of this constructor, but since std::to_address is a noexcept function, this constructor throws if and only when end - begin throws, we should add a *Throws* element for it.

[2021-08-20; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 21.4.3.2 [string.view.cons] as indicated:

```
template<class It, class End>
  constexpr basic_string_view(It begin, End end);
```

-7- *Constraints:*

[…]

-8- *Preconditions:*

[…]

-9- *Effects:* Initializes data_ with to_address(begin) and initializes size_ with end - begin.

-?- *Throws:* When and what end - begin throws.

3574. `common_iterator` should be completely `constexpr`-able

Section: 23.5.4.1 [common.iterator] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2021-07-21 **Last modified:** 2021-08-23

Priority: 3

View all other issues in [common.iterator].

Discussion:

After P2231R1, `variant` becomes completely `constexpr`-able, which means that `common_iterator` can also be completely `constexpr`-able.

[2021-08-20; Reflector poll]

Set priority to 3 after reflector poll.

[2021-08-23; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 23.5.4.1 [common.iterator], class template `common_iterator` synopsis, as indicated:

```
namespace std {
  template<input_or_output_iterator I, sentinel_for<I> S>
    requires (!same_as<I, S> && copyable<I>)
  class common_iterator {
  public:
    constexpr common_iterator() requires default_initializable<I> = default;
    constexpr common_iterator(I i);
    constexpr common_iterator(S s);
    template<class I2, class S2>
      requires convertible_to<const I2&, I> && convertible_to<const S2&, S>
        constexpr common_iterator(const common_iterator<I2, S2>& x);

    template<class I2, class S2>
      requires convertible_to<const I2&, I> && convertible_to<const S2&, S> &&
               assignable_from<I&, const I2&> && assignable_from<S&, const S2&>
      constexpr common_iterator& operator=(const common_iterator<I2, S2>& x);

    constexpr decltype(auto) operator*();
    constexpr decltype(auto) operator*() const
      requires dereferenceable<const I>;
    constexpr decltype(auto) operator->() const
      requires see below;

    constexpr common_iterator& operator++();
    constexpr decltype(auto) operator++(int);

    template<class I2, sentinel_for<I> S2>
      requires sentinel_for<S, I2>
    friend constexpr bool operator==(
      const common_iterator& x, const common_iterator<I2, S2>& y);
    template<class I2, sentinel_for<I> S2>
      requires sentinel_for<S, I2> && equality_comparable_with<I, I2>
    friend constexpr bool operator==(
      const common_iterator& x, const common_iterator<I2, S2>& y);

    template<sized_sentinel_for<I> I2, sized_sentinel_for<I> S2>
      requires sized_sentinel_for<S, I2>
    friend constexpr iter_difference_t<I2> operator-(
      const common_iterator& x, const common_iterator<I2, S2>& y);

    friend constexpr iter_rvalue_reference_t<I> iter_move(const common_iterator& i)
      noexcept(noexcept(ranges::iter_move(declval<const I&>())))
        requires input_iterator<I>;
    template<indirectly_swappable<I> I2, class S2>
      friend constexpr void iter_swap(const common_iterator& x, const common_iterator<I2, S2>& y)
        noexcept(noexcept(ranges::iter_swap(declval<const I&>(), declval<const I2&>())));

  private:
    variant<I, S> v_; // exposition only
  };
  […]
}
```

2. Modify 23.5.4.3 [common.iter.const] as indicated:

```
template<class I2, class S2>
  requires convertible_to<const I2&, I> && convertible_to<const S2&, S> &&
           assignable_from<I&, const I2&> && assignable_from<S&, const S2&>
    constexpr common_iterator& operator=(const common_iterator<I2, S2>& x);
```

-5- *Preconditions:* `x.v_.valueless_by_exception()` is `false`.

[…]

3. Modify 23.5.4.4 [common.iter.access] as indicated:

```
constexpr decltype(auto) operator*();
constexpr decltype(auto) operator*() const
  requires dereferenceable<const I>;
```

-1- *Preconditions:* `holds_alternative<I>(v_)` is `true`.

[…]

```
constexpr decltype(auto) operator->() const
  requires see below;
```

-3- The expression in the *requires-clause* is equivalent to:

[…]

4. Modify 23.5.4.5 [common.iter.nav] as indicated:

```
constexpr common_iterator& operator++();
```

-1- *Preconditions:* `holds_alternative<I>(v_)` is `true`.

[…]

```
constexpr decltype(auto) operator++(int);
```

-4- *Preconditions:* `holds_alternative<I>(v_)` is `true`.

[…]

5. Modify 23.5.4.6 [common.iter.cmp] as indicated:

```
template<class I2, sentinel_for<I> S2>
  requires sentinel_for<S, I2>
friend constexpr bool operator==(
  const common_iterator& x, const common_iterator<I2, S2>& y);
```

-1- *Preconditions:* […]

```
template<class I2, sentinel_for<I> S2>
  requires sentinel_for<S, I2> && equality_comparable_with<I, I2>
friend constexpr bool operator==(
  const common_iterator& x, const common_iterator<I2, S2>& y);
```

-3- *Preconditions:* […]

```
template<sized_sentinel_for<I> I2, sized_sentinel_for<I> S2>
  requires sized_sentinel_for<S, I2>
friend constexpr iter_difference_t<I2> operator-(
  const common_iterator& x, const common_iterator<I2, S2>& y);
```

-5- *Preconditions:* […]

6. Modify 23.5.4.7 [common.iter.cust] as indicated:

```
friend constexpr iter_rvalue_reference_t<I> iter_move(const common_iterator& i)
  noexcept(noexcept(ranges::iter_move(declval<const I&>())))
    requires input_iterator<I>;
```

-1- *Preconditions:* […]

```
template<indirectly_swappable<I> I2, class S2>
  friend constexpr void iter_swap(const common_iterator& x, const common_iterator<I2, S2>& y)
    noexcept(noexcept(ranges::iter_swap(declval<const I&>(), declval<const I2&>())));
```

-3- *Preconditions:* […]

3580. `iota_view`'s `iterator`'s binary `operator+` should be improved

Section: 24.6.4.3 [range.iota.iterator] **Status:** Tentatively Ready **Submitter:** Zoe Carver **Opened:** 2021-08-14 **Last modified:** 2021-08-20

Priority: Not Prioritized

View all other issues in [range.iota.iterator].

Discussion:

`iota_view`'s iterator's `operator+` could avoid a copy construct by doing "`i += n; return i`" rather than "`return i += n;`" as seen in 24.6.4.3 [range.iota.iterator].

This is what libc++ has implemented and shipped, even though it may not technically be conforming (if a program asserted the number of copies, for example).

It might be good to update this operator and the minus operator accordingly.

[2021-08-20; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 24.6.4.3 [range.iota.iterator] as indicated:

```
friend constexpr iterator operator+(iterator i, difference_type n)
  requires advanceable<W>;
```

-20- *Effects:* Equivalent to:

```
~~return~~ i += n;
return i;
```

```
friend constexpr iterator operator+(difference_type n, iterator i)
  requires advanceable<W>;
```

-21- *Effects:* Equivalent to: `return i + n;`

```
friend constexpr iterator operator-(iterator i, difference_type n)
  requires advanceable<W>;
```

-22- *Effects:* Equivalent to:

```
~~return~~ i -= n;
return i;
```

3581. The range constructor makes `basic_string_view` not trivially move constructible

Section: 21.4.3.2 [string.view.cons] **Status:** Tentatively Ready **Submitter:** Jiang An, Casey Carter **Opened:** 2021-08-16 **Last modified:** 2021-09-20

Priority: Not Prioritized

View other active issues in [string.view.cons].

View all other issues in [string.view.cons].

Discussion:

Jiang An:

The issue is found in this Microsoft STL pull request.

I think an additional constraint should be added to 21.4.3.2 [string.view.cons]/11 (a similar constraint is already present for `reference_wrapper`):

(11.?) — `is_same_v<remove_cvref_t<R>, basic_string_view>` is `false`.

Casey Carter:

P1989R2 "Range constructor for `std::string_view` 2: Constrain Harder" added a converting constructor to `basic_string_view` that accepts (by forwarding reference) any sized contiguous range with a compatible element type. This constructor unfortunately intercepts attempts at move construction which previously would have resulted in a call to the copy constructor. (I suspect the authors of P1989 were under the mistaken impression that `basic_string_view` had a defaulted move constructor, which would sidestep this issue by being chosen by overload resolution via the "non-template vs. template" tiebreaker.)

The resulting inefficiency could be corrected by adding a defaulted move constructor and move assignment operator to `basic_string_view`, but it would be awkward to add text to specify that these moves always leave the source intact. Presumably this is why the move operations were omitted in the first place. We therefore recommend constraining the conversion constructor template to reject arguments whose decayed type is `basic_string_view` (which we should probably do for all conversion constructor templates in the Standard Library).

Implementation experience: MSVC STL is in the process of implementing this fix. Per Jonathan Wakely, libstdc++ has a similar constraint.

[2021-09-20; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 21.4.3.2 [string.view.cons] as indicated:

```
template<class R>
  constexpr basic_string_view(R&& r);
```

-10- Let `d` be an lvalue of type `remove_cvref_t<R>`.

-11- *Constraints*:

(11.?) — `remove_cvref_t<R>` is not the same type as `basic_string_view`,

(11.1) — `R` models `ranges::contiguous_range` and `ranges::sized_range`,

(11.2) — `is_same_v<ranges::range_value_t<R>, charT>` is `true`,

[…]

3585. Variant converting assignment with immovable alternative

Section: 20.7.3.4 [variant.assign] **Status:** Tentatively Ready **Submitter:** Barry Revzin **Opened:** 2021-09-01 **Last modified:** 2021-09-20

Priority: Not Prioritized

View other active issues in [variant.assign].

View all other issues in [variant.assign].

Discussion:

Example originally from StackOverflow but with a more reasonable example from Tim Song:

```
#include <variant>
#include <string>

struct A {
  A() = default;
  A(A&&) = delete;
};

int main() {
  std::variant<A, std::string> v;
  v = "hello";
}
```

There is implementation divergence here: libstdc++ rejects, libc++ and msvc accept.

20.7.3.4 [variant.assign] bullet (13.3) says that if we're changing the alternative in assignment and it is not the case that the converting construction won't throw (as in the above), then "Otherwise, equivalent to `operator=(variant(std::forward<T>(t)))`." That is, we defer to move assignment.

`variant<A, string>` isn't move-assignable (because `A` isn't move constructible). Per the wording in the standard, we have to reject this. libstdc++ follows the wording of the standard.

But we don't actually have to do a full move assignment here, since we know the situation we're in is changing the alternative, so the fact that `A` isn't move-assignable shouldn't matter. libc++ and msvc instead do something more direct, allowing the above program.

[2021-09-20; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 20.7.3.4 [variant.assign] as indicated:

[*Drafting note:* This should cover the case that we want to cover: that if construction of T_j from `T` throws, we haven't yet changed any state in the variant.]

`template<class T> constexpr variant& operator=(T&& t) noexcept(`*see below*`);`

-11- Let T_j be a type that is determined as follows: build an imaginary function *FUN*(T_i) for each alternative type T_i for which T_i `x[] = {std::forward<T>(t)};` is well-formed for some invented variable `x`. The overload *FUN*(T_j) selected by overload resolution for the expression *FUN*(`std::forward<T>(t)`) defines the alternative T_j which is the type of the contained value after assignment.

-12- *Constraints:* […]

-13- *Effects:*

(13.1) — If `*this` holds a T_j, assigns `std::forward<T>(t)` to the value contained in `*this`.

(13.2) — Otherwise, if `is_nothrow_constructible_v<`T_j`, T> || !is_nothrow_move_constructible_v<`T_j`>` is `true`, equivalent to `emplace<`*j*`>(std::forward<T>(t))`.

(13.3) — Otherwise, equivalent to ~~`operator=(variant(std::forward<T>(t)))`~~`emplace<`*j*`>(`T_j`(std::forward<T>(t)))`.

3589. The `const` lvalue reference overload of `get` for `subrange` does not constrain `I` to be `copyable` when `N == 0`

Section: 24.5.4 [range.subrange] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2021-09-08 **Last modified:** 2021-09-20

Priority: 3

View other active issues in [range.subrange].

View all other issues in [range.subrange].

Discussion:

The `const` lvalue reference overload of `get` used for `subrange` only constraint that `N < 2`, this will cause the "discards qualifiers" error inside the function body when applying `get` to the lvalue `subrange` that stores a non-`copyable` iterator since its `begin()` is non-`const` qualified, we probably need to add a constraint for it.

[2021-09-20; Reflector poll]

Set priority to 3 after reflector poll.

[2021-09-20; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 24.5.4 [range.subrange] as indicated:

```
namespace std::ranges {
  […]
  template<size_t N, class I, class S, subrange_kind K>
    requires ((N ~~< 2~~== 0 && copyable<I>) || N == 1)
    constexpr auto get(const subrange<I, S, K>& r);

  template<size_t N, class I, class S, subrange_kind K>
    requires (N < 2)
    constexpr auto get(subrange<I, S, K>&& r);
}
```

2. Modify 24.5.4.3 [range.subrange.access] as indicated:

```
  […]
  template<size_t N, class I, class S, subrange_kind K>
    requires ((N ~~< 2~~== 0 && copyable<I>) || N == 1)
    constexpr auto get(const subrange<I, S, K>& r);
  template<size_t N, class I, class S, subrange_kind K>
    requires (N < 2)
    constexpr auto get(subrange<I, S, K>&& r);
```

-10- *Effects*: Equivalent to:

```
if constexpr (N == 0)
  return r.begin();
else
  return r.end();
```

3590. `split_view::base() const &` is overconstrained

Section: 24.7.14.2 [range.split.view] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-09-13 **Last modified:** 2021-09-24

Priority: Not Prioritized

Discussion:

Unlike every other range adaptor, `split_view::base() const &` is constrained on `copyable<V>` instead of `copy_constructible<V>`.

Since this function just performs a copy construction, there is no reason to require all of `copyable` — `copy_constructible` is sufficient.

[2021-09-24; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 24.7.14.2 [range.split.view], class template `split_view` synopsis, as indicated:

```
namespace std::ranges {
  template<forward_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to>
  class split_view : public view_interface<split_view<V, Pattern>> {
  private:
    […]
  public:
    […]

    constexpr V base() const& requires ~~copyable~~copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    […]
  };
```

3591. `lazy_split_view<input_view>::inner-iterator::base() &&` invalidates outer iterators

Section: 24.7.13.5 [range.lazy.split.inner] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-09-13 **Last modified:** 2021-09-24

Priority: Not Prioritized

Discussion:

The `base() &&` members of iterator adaptors (and iterators of range adaptors) invalidate the adaptor itself by moving from the contained iterator. This is generally unobjectionable — since you are calling `base()` on an rvalue, it is expected that you won't be using it afterwards.

But `lazy_split_view<input_view>::inner-iterator::base() &&` is special: the iterator being moved from is stored in the `lazy_split_view` itself and shared between the inner and outer iterators, so the operation invalidates not just the *`inner-iterator`* on which it is called, but also the *`outer-iterator`* from which the *`inner-iterator`* was obtained. This spooky-action-at-a-distance behavior can be surprising, and the value category of the inner iterator seems to be too subtle to base it upon.

The PR below constrains this overload to forward ranges. Forward iterators are copyable anyway, but moving could potentially be more efficient.

[2021-09-24; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 24.7.13.5 [range.lazy.split.inner] as indicated:

    [*Drafting note:* The constraint uses `forward_range<V>` since that's the condition for caching in `lazy_split_view`.]

```
namespace std::ranges {
  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
             (forward_range<V> || tiny-range<Pattern>)
  template<bool Const>
  struct lazy_split_view<V, Pattern>::inner-iterator {
  private:
    […]
  public:
    […]

    constexpr const iterator_t<Base>& base() const &;
    constexpr iterator_t<Base> base() && requires forward_range<V>;

    […]
  };

[…]

constexpr iterator_t<Base> base() && requires forward_range<V>;
```

-4- *Effects:* Equivalent to: `return std::move(i_.current);`

3592. `lazy_split_view` needs to check the simpleness of `Pattern`

Section: 24.7.13.2 [range.lazy.split.view] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-09-15 **Last modified:** 2021-09-24

Priority: Not Prioritized

View other active issues in [range.lazy.split.view].

View all other issues in [range.lazy.split.view].

Discussion:

For forward ranges, the non-`const` versions of `lazy_split_view::begin()` and `lazy_split_view::end()` returns an *`outer-iterator`*`<`*`simple-view`*`<V>>`, promoting to `const` if `V` models *`simple-view`*. However, this failed to take `Pattern` into account, even though that will also be accessed as `const` if `const`-promotion takes place. There is no requirement that `const Pattern` is a range, however.

[2021-09-24; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 24.7.13.2 [range.lazy.split.view], class template `lazy_split_view` synopsis, as indicated:

```
namespace std::ranges {

  […]

  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
            (forward_range<V> || tiny-range<Pattern>)
  class lazy_split_view : public view_interface<lazy_split_view<V, Pattern>> {
  private:
    […]
  public:
    […]

    constexpr auto begin() {
      if constexpr (forward_range<V>)
        return outer-iterator<simple-view<V> &&
          simple-view<Pattern>>{*this, ranges::begin(base_)};
      else {
        current_ = ranges::begin(base_);
        return outer-iterator<false>{*this};
      }
    }

    […]

    constexpr auto end() requires forward_range<V> && common_range<V> {
      return outer-iterator<simple-view<V> &&
        simple-view<Pattern>>{*this, ranges::end(base_)};
    }
```

```
    […]
  };

  […]

}
```

3593. Several iterators' `base() const &` and `lazy_split_view::`*`outer-iterator`*`::value_type::end()` missing `noexcept`

Section: 23.5.3 [move.iterators], 23.5.6 [iterators.counted], 24.7.6.3 [range.filter.iterator], 24.7.7.3 [range.transform.iterator], 24.7.13.4 [range.lazy.split.outer.value], 24.7.13.5 [range.lazy.split.inner], 24.7.18.3 [range.elements.iterator] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2021-09-14 **Last modified:** 2021-09-24

Priority: Not Prioritized

View all other issues in [move.iterators].

Discussion:

LWG 3391 and 3533 changed some iterators' `base() const &` from returning value to returning `const` reference, which also prevents them from throwing exceptions, we should add `noexcept` for them. Also, `lazy_split_view::`*`outer-iterator`*`::value_type::end()` can be `noexcept` since it only returns `default_sentinel`.

[2021-09-24; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 23.5.3 [move.iterators], class template `move_iterator` synopsis, as indicated:

```
[…]
constexpr const iterator_type& base() const & noexcept;
constexpr iterator_type base() &&;
[…]
```

2. Modify 23.5.3.5 [move.iter.op.conv] as indicated:

```
constexpr const Iterator& base() const & noexcept;
```

-1- *Returns:* `current`.

3. Modify 23.5.6.1 [counted.iterator], class template `counted_iterator` synopsis, as indicated:

```
[…]
constexpr const I& base() const & noexcept;
constexpr I base() &&;
[…]
```

4. Modify 23.5.6.3 [counted.iter.access] as indicated:

```
constexpr const I& base() const & noexcept;
```

-1- *Effects:* Equivalent to: `return current;`

5. Modify 24.7.6.3 [range.filter.iterator] as indicated:

```
[…]
constexpr const iterator_t<V>& base() const & noexcept;
constexpr iterator_t<V> base() &&;
[…]
```

[…]

```
constexpr const iterator_t<V>& base() const & noexcept;
```

-5- *Effects:* Equivalent to: `return `*`current_`*`;`

6. Modify 24.7.7.3 [range.transform.iterator] as indicated:

```
[…]
constexpr const iterator_t<Base>& base() const & noexcept;
constexpr iterator_t<Base> base() &&;
[…]
```

[…]

```
constexpr const iterator_t<Base>& base() const & noexcept;
```

-5- *Effects:* Equivalent to: `return current_;`

7. Modify 24.7.13.4 [range.lazy.split.outer.value] as indicated:

```
[…]
constexpr inner-iterator<Const> begin() const;
constexpr default_sentinel_t end() const noexcept;
[…]
```

[…]

```
constexpr default_sentinel_t end() const noexcept;
```

-3- *Effects:* Equivalent to: `return default_sentinel;`

8. Modify 24.7.13.5 [range.lazy.split.inner] as indicated:

```
[…]
constexpr const iterator_t<Base>& base() const & noexcept;
constexpr iterator_t<Base> base() &&;
[…]
```

[…]

```
constexpr const iterator_t<Base>& base() const & noexcept;
```

-3- *Effects:* Equivalent to: `return i_.current;`

9. Modify 24.7.18.3 [range.elements.iterator] as indicated:

```
[…]
constexpr const iterator_t<Base>& base() const& noexcept;
constexpr iterator_t<Base> base() &&;
[…]
```

[…]

```
constexpr const iterator_t<Base>& base() const& noexcept;
```

-6- *Effects:* Equivalent to: `return current_;`

3595. Exposition-only classes *proxy* and *postfix-proxy* for common_iterator should be fully constexpr

Section: 23.5.4.4 [common.iter.access], 23.5.4.5 [common.iter.nav] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2021-09-18 **Last modified:** 2021-09-24

Priority: Not Prioritized

Discussion:

LWG 3574 added `constexpr` to all member functions and friends of `common_iterator` to make it fully `constexpr`, but accidentally omitted the exposition-only classes *`proxy`* and *`postfix-proxy`* defined for some member functions. We should make these two classes fully `constexpr`, too.

[2021-09-24; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4892.

1. Modify 23.5.4.4 [common.iter.access] as indicated:

```
decltype(auto) operator->() const
  requires see below;
```

-3- […]

-4- […]

-5- *Effects:*

(5.1) — […]

(5.2) — […]

(5.3) — Otherwise, equivalent to: `return proxy(*get<I>(v_));` where *proxy* is the exposition-only class:

```
class proxy {
  iter_value_t<I> keep_;
  constexpr proxy(iter_reference_t<I>&& x)
    : keep_(std::move(x)) {}
public:
  constexpr const iter_value_t<I>* operator->() const noexcept {
    return addressof(keep_);
  }
};
```

2. Modify 23.5.4.5 [common.iter.nav] as indicated:

```
decltype(auto) operator++(int);
```

-4- […]

-5- *Effects:* If I models forward_iterator, equivalent to:

[…]

Otherwise, if […]

[…]

Otherwise, equivalent to:

```
postfix-proxy p(**this);
++*this;
return p;
```

where *postfix-proxy* is the exposition-only class:

```
class postfix-proxy {
  iter_value_t<I> keep_;
  constexpr postfix-proxy(iter_reference_t<I>&& x)
    : keep_(std::forward<iter_reference_t<I>>(x)) {}
public:
  constexpr const iter_value_t<I>& operator*() const noexcept {
    return keep_;
  }
};
```